The Concise TypeScript Book

Simone Poggiali

Cuốn sách TypeScript súc tích

Cuốn sách TypeScript súc tích cung cấp cái nhìn tổng quan toàn diện và ngắn gọn về các khả năng của TypeScript. Sách đưa ra những giải thích rõ ràng bao quát mọi khía cạnh của phiên bản ngôn ngữ mới nhất, từ hệ thống kiểu mạnh mẽ đến các tính năng nâng cao.

Dù bạn là người mới bắt đầu hay một lập trình viên giàu kinh nghiệm, cuốn sách này là một tài liệu vô cùng giá trị giúp nâng cao hiểu biết và kỹ năng sử dụng TypeScript.

Cuốn sách này hoàn toàn miễn phí và mã nguồn mở.

Tôi tin rằng giáo dục kỹ thuật chất lượng cao nên dễ dàng tiếp cận với mọi người. Vì lý do này, tôi cung cấp cuốn sách miễn phí và thường xuyên cập nhật với các cải tiến cùng những ví dụ mới.

Khám phá **The Concise TypeScript Book Plus Edition**.



Dành cho những độc giả muốn đi xa hơn phiên bản mã nguồn mở, **The Concise TypeScript Book Plus Edition: React and Real-World Patterns for TypeScript 7** bao gồm nội dung bổ sung và độc quyền, tập trung vào việc áp dụng thực tế.

Plus Edition bao gồm:

- **Được cập nhật cho TypeScript 7** — bao quát các tính năng TypeScript 7 mới nhất và những cải tiến của ngôn ngữ.
- **TypeScript với React** — hướng dẫn thực tế về cách định kiểu cho component, props, hooks, events, children, refs và các mẫu React phổ biến.

- **Công thức TypeScript cho dự án thực tế** — các ví dụ tập trung giải quyết những vấn đề thực tế mà lập trình viên gặp phải khi xây dựng và bảo trì ứng dụng TypeScript.

Khi mua Plus Edition, bạn cũng trực tiếp hỗ trợ việc tiếp tục phát triển và duy trì cuốn sách miễn phí và mã nguồn mở.

Plus Edition hiện có bằng tiếng Anh và tiếng Ý trên Amazon toàn cầu. Khám phá Plus Edition và mua trên Amazon.

Hỗ trợ dự án

Nếu cuốn sách miễn phí đã giúp bạn sửa một lỗi, hiểu một khái niệm khó hoặc tiến xa hơn trong sự nghiệp, hãy cân nhắc hỗ trợ công việc của tôi bằng cách đóng góp tùy theo khả năng, với mức gợi ý là **$5**, hoặc mời tôi một ly cà phê.

Sự hỗ trợ của bạn giúp tôi giữ nội dung luôn cập nhật và mở rộng sách bằng các ví dụ mới, giải thích rõ ràng hơn và thêm hướng dẫn thực tế.





Các bản dịch

Cuốn sách này đã được dịch sang một số ngôn ngữ, bao gồm:

Tiếng Anh

Tiếng Bulgaria

Tiếng Đức

Tiếng Pháp

Tiếng Indonesia

Tiếng Ý

Tiếng Nhật

Tiếng Hàn

Tiếng Ba Lan

Tiếng Bồ Đào Nha (Brazil)

Tiếng Thụy Điển

Tiếng Thổ Nhĩ Kỳ

Tiếng Việt

Tiếng Trung

Tiếng Tây Ban Nha

Tiếng Thái

Tiếng Nga

Tiếng Ả Rập Tiếng Séc

Tải xuống và trang web

Bạn cũng có thể tải xuống phiên bản EPUB:

https://github.com/gibbok/typescript-book/tree/main/downloads

Phiên bản trực tuyến có tại:

https://gibbok.github.io/typescript-book

Mục lục

- Cuốn sách TypeScript súc tích
  - Hỗ trợ dự án
  - Các bản dịch
  - Tải xuống và trang web
  - Mục lục
  - Giới thiệu
  - Về tác giả
  - Giới thiệu về TypeScript
    - TypeScript là gì?
    - Tại sao dùng TypeScript?
    - TypeScript và JavaScript
    - Sinh mã TypeScript
    - JavaScript hiện đại ngay bây giờ (Downleveling)
  - Bắt đầu với TypeScript
    - Cài đặt
    - Cấu hình
    - Tệp cấu hình TypeScript
      - target
      - lib
      - strict
      - module
      - moduleResolution
      - esModuleInterop
      - jsx

        - skipLibCheck
        - files
        - include
        - exclude
    - importHelpers
    - Lời khuyên khi chuyển sang TypeScript
- Khám phá hệ thống kiểu
    - Dịch vụ ngôn ngữ TypeScript
    - Kiểu cấu trúc
    - Các quy tắc so sánh cơ bản của TypeScript
    - Kiểu dưới dạng tập hợp
    - Gán kiểu: Khai báo kiểu và Type Assertion
        - Khai báo kiểu
        - Type Assertion
        - Khai báo ambient
    - Kiểm tra thuộc tính và kiểm tra thuộc tính dư thừa
    - Kiểu yếu
    - Kiểm tra nghiêm ngặt object literal (Freshness)
    - Suy luận kiểu
    - Suy luận nâng cao hơn
    - Type Widening
    - Const
        - Modifier Const trên tham số kiểu
        - Const assertion
    - Chú thích kiểu tường minh
    - Thu hẹp kiểu
        - Điều kiện
        - Throw hoặc return
        - Discriminated Union
        - Type Guard do người dùng định nghĩa
        - Thu hẹp bằng switch-true
- Các kiểu nguyên thủy

        - string
        - boolean
        - number
        - bigint
        - Symbol
        - null và undefined
        - Array
        - any
    - Chú thích kiểu
    - Thuộc tính tùy chọn
    - Thuộc tính readonly
    - Index Signature
    - Mở rộng kiểu
    - Literal Type
    - Suy luận literal
    - strictNullChecks
    - Enum
        - Enum số
        - Enum chuỗi
        - Const enum
        - Ánh xạ ngược
        - Ambient enum
        - Thành viên computed và constant
    - Narrowing
        - Type guard typeof
        - Thu hẹp theo truthiness
        - Thu hẹp theo phép so sánh bằng
        - Thu hẹp bằng toán tử In
        - Thu hẹp bằng instanceof
    - Phép gán
    - Phân tích luồng điều khiển
    - Type Predicate

- Discriminated Union
- Kiểu never
- Kiểm tra tính đầy đủ
- Kiểu đối tượng
- Tuple Type (ẩn danh)
- Named Tuple Type (có nhãn)
- Tuple có độ dài cố định
- Union Type
- Intersection Type
- Type Indexing
- Kiểu từ giá trị
- Kiểu từ giá trị trả về của hàm
- Kiểu từ module
- Mapped Type
- Modifier của Mapped Type
- Conditional Type
- Distributive Conditional Type
- Suy luận kiểu infer trong Conditional Type
- Conditional Type được định nghĩa sẵn
- Template Union Type
- Kiểu any
- Kiểu unknown
- Kiểu void
- Kiểu dữ liệu never
- Interface và Type
  - Cú pháp chung
  - Kiểu cơ bản
  - Đối tượng và Interface
  - Union Type và Intersection Type
- Primitive Type tích hợp
- Các đối tượng JS tích hợp phổ biến
- Overload

- Hợp nhất và mở rộng
- Khác biệt giữa Type và Interface
- Class
  - Cú pháp class thông dụng
  - Constructor
  - Constructor private và protected
  - Access Modifier
  - Get và Set
  - Auto-Accessor trong Class
  - this
  - Parameter Property
  - Abstract Class
  - Với Generic
  - Decorator
    - Class Decorator
    - Property Decorator
    - Method Decorator
    - Getter và Setter Decorator
    - Metadata của Decorator
  - Kế thừa
  - Thành viên static
  - Khởi tạo thuộc tính
  - Method overloading
- Generic
  - Generic Type
  - Generic Class
  - Ràng buộc Generic
  - Thu hẹp theo ngữ cảnh cho Generic
- Kiểu cấu trúc bị xóa
- Namespace
- Symbol
- Triple-Slash Directive

- Thao tác kiểu
  - Tạo kiểu từ kiểu
  - Indexed Access Type
  - Utility Type
    - Awaited<T>
    - Partial<T>
    - Required<T>
    - Readonly<T>
    - Record<K, T>
    - Pick<T, K>
    - Omit<T, K>
    - Exclude<T, U>
    - Extract<T, U>
    - NonNullable<T>
    - Parameters<T>
    - ConstructorParameters<T>
    - ReturnType<T>
    - InstanceType<T>
    - ThisParameterType<T>
    - OmitThisParameter<T>
    - ThisType<T>
    - Uppercase<T>
    - Lowercase<T>
    - Capitalize<T>
    - Uncapitalize<T>
    - NoInfer<T>
- Khác
  - Lỗi và xử lý ngoại lệ
  - Mixin class
  - Các tính năng ngôn ngữ bất đồng bộ
  - Iterator và Generator
  - Tham khảo TsDocs JSDoc

- @types
- JSX
- ES6 Module
- Toán tử lũy thừa ES7
- Câu lệnh for-await-of
- Meta-property new target
- Biểu thức Dynamic Import
- “tsc –watch”
- Toán tử Non-null Assertion
- Khai báo có giá trị mặc định
- Optional Chaining
- Toán tử Nullish Coalescing
- Template Literal Type
- Function overloading
- Kiểu đệ quy
- Recursive Conditional Type
- Hỗ trợ ECMAScript Module trong Node
- Assertion Function
- Variadic Tuple Type
- Boxed type
- Covariance và Contravariance trong TypeScript
  - Variance Annotation tùy chọn cho tham số kiểu
- Template String Pattern Index Signature
- Toán tử satisfies
- Import và Export chỉ dành cho Type
- Khai báo using và Explicit Resource Management
  - Khai báo await using
- Import Attribute
- Kiểm tra cú pháp Regular Expression
- import defer

Giới thiệu

Chào mừng bạn đến với Cuốn sách TypeScript súc tích! Hướng dẫn này trang bị cho bạn kiến thức thiết yếu và kỹ năng thực tế để phát triển TypeScript hiệu quả. Hãy khám phá các khái niệm và kỹ thuật quan trọng để viết mã sạch và vững chắc. Dù bạn là người mới bắt đầu hay một lập trình viên giàu kinh nghiệm, cuốn sách này vừa là một hướng dẫn toàn diện vừa là tài liệu tham khảo tiện dụng để tận dụng sức mạnh của TypeScript trong các dự án của bạn.

Cuốn sách này bao quát TypeScript 7.0.

Về tác giả

Simone Poggiali là một Staff Engineer giàu kinh nghiệm, đam mê viết mã ở tiêu chuẩn chuyên nghiệp từ những năm 90. Trong suốt sự nghiệp quốc tế của mình, ông đã đóng góp cho nhiều dự án của nhiều khách hàng khác nhau, từ startup đến các tổ chức lớn. Những công ty nổi bật như HelloFresh, Siemens, O2, Leroy Merlin và Snowplow đã hưởng lợi từ chuyên môn và sự tận tâm của ông.

Bạn có thể liên hệ với Simone Poggiali qua các nền tảng sau:

- LinkedIn: https://www.linkedin.com/in/simone-poggiali
- GitHub: https://github.com/gibbok
- Bluesky: https://bsky.app/profile/gibbok.bsky.social
- Email: gibbok.coding gmail.com

Danh sách đầy đủ những người đóng góp: https://github.com/gibbok/typescript-book/graphs/contributors

Giới thiệu về TypeScript

TypeScript là gì?

TypeScript là một ngôn ngữ lập trình có hệ thống kiểu mạnh, được xây dựng dựa trên JavaScript. Ban đầu ngôn ngữ này do Anders Hejlsberg thiết kế vào năm 2012 và hiện được Microsoft phát triển và duy trì như một dự án mã nguồn mở.

TypeScript được biên dịch thành JavaScript và có thể chạy trong bất kỳ môi trường thực thi JavaScript nào (ví dụ: trình duyệt hoặc Node.js trên máy chủ).

Ngôn ngữ này hỗ trợ nhiều mô hình lập trình như lập trình hàm, generic, mệnh lệnh và hướng đối tượng, đồng thời là một ngôn ngữ được biên dịch (chuyển dịch) sang JavaScript trước khi thực thi.

Tại sao dùng TypeScript?

TypeScript là một ngôn ngữ có hệ thống kiểu mạnh, giúp ngăn ngừa các lỗi lập trình phổ biến và tránh một số loại lỗi thời gian chạy trước khi chương trình được thực thi.

Một ngôn ngữ có hệ thống kiểu mạnh cho phép lập trình viên chỉ định nhiều ràng buộc và hành vi của chương trình trong các định nghĩa kiểu dữ liệu, nhờ đó dễ dàng xác minh tính đúng đắn của phần mềm và ngăn ngừa lỗi. Điều này đặc biệt có giá trị trong các ứng dụng quy mô lớn.

Một số lợi ích của TypeScript:

- Kiểu tĩnh, có thể sử dụng hệ thống kiểu mạnh

- Suy luận kiểu
- Truy cập các tính năng ES6 và ES7
- Tương thích đa nền tảng và đa trình duyệt
- Hỗ trợ công cụ với IntelliSense

TypeScript và JavaScript

TypeScript được viết trong các tệp `.ts` hoặc `.tsx`, trong khi các tệp JavaScript được viết trong `.js` hoặc `.jsx`.

Các tệp có phần mở rộng `.tsx` hoặc `.jsx` có thể chứa JavaScript Syntax Extension JSX, được sử dụng trong React để phát triển giao diện người dùng.

Về cú pháp, TypeScript là một tập cha có kiểu của JavaScript (ECMAScript 2015). Mọi mã JavaScript đều là mã TypeScript hợp lệ, nhưng chiều ngược lại không phải lúc nào cũng đúng.

Ví dụ, hãy xem một hàm trong tệp JavaScript có phần mở rộng `.js`, như sau:

```
const sum = (a, b) => a + b;
```

Có thể chuyển đổi và sử dụng hàm này trong TypeScript bằng cách đổi phần mở rộng tệp thành `.ts`. Tuy nhiên, nếu cùng hàm đó được chú thích bằng các kiểu TypeScript, nó không thể chạy trong bất kỳ môi trường thực thi JavaScript nào nếu chưa được biên dịch. Mã TypeScript sau sẽ gây lỗi cú pháp nếu không được biên dịch:

```
const sum = (a: number, b: number): number => a + b;
```

TypeScript được thiết kế để phát hiện các lỗi thời gian chạy tiềm ẩn ngay tại thời điểm biên dịch bằng cách cho phép lập trình

viên biểu đạt ý định thông qua chú thích kiểu. Ngoài ra, nhờ suy luận kiểu, TypeScript cũng có thể phát hiện một số vấn đề ngay cả khi không có chú thích kiểu tường minh. Ví dụ, đoạn mã sau không chỉ định bất kỳ kiểu TypeScript nào:

```
const items = [{ x: 1 }, { x: 2 }];
const result = items.filter(item => item.y);
```

Trong trường hợp này, TypeScript phát hiện lỗi và báo:

```
Property 'y' does not exist on type '{ x: number; }'.
```

Hệ thống kiểu của TypeScript chịu ảnh hưởng lớn từ hành vi thời gian chạy của JavaScript. Ví dụ, toán tử cộng (+), trong JavaScript có thể thực hiện nối chuỗi hoặc cộng số, cũng được mô hình hóa tương tự trong TypeScript:

```
const result = '1' + 1; // Kết quả có kiểu string
```

Nhóm phát triển TypeScript đã chủ động quyết định đánh dấu các cách sử dụng JavaScript bất thường là lỗi. Ví dụ, hãy xem đoạn mã JavaScript hợp lệ sau:

```
const result = 1 + true; // Trong JavaScript, kết quả bằng 2
```

Tuy nhiên, TypeScript báo lỗi:

```
Operator '+' cannot be applied to types 'number' and 'boolean'.
```

Lỗi này xảy ra vì TypeScript thực thi nghiêm ngặt tính tương thích kiểu và trong trường hợp này xác định phép toán giữa một số và một giá trị boolean là không hợp lệ.

Sinh mã TypeScript

Trình biên dịch TypeScript có hai trách nhiệm chính: kiểm tra lỗi kiểu và biên dịch sang JavaScript. Hai quá trình này độc lập với nhau. Kiểu không ảnh hưởng đến việc thực thi mã trong môi trường JavaScript vì chúng bị xóa hoàn toàn trong quá trình biên dịch. TypeScript vẫn có thể xuất JavaScript ngay cả khi có lỗi kiểu. Dưới đây là ví dụ về mã TypeScript có lỗi kiểu:

```
const add = (a: number, b: number): number => a + b;
const result = add('x', 'y'); // Đối số có kiểu 'string' không thể gán cho tham số có kiểu 'number'.
```

Tuy nhiên, nó vẫn có thể tạo ra đầu ra JavaScript có thể thực thi:

```
'use strict';
const add = (a, b) => a + b;
const result = add('x', 'y'); // xy
```

Không thể kiểm tra các kiểu TypeScript tại thời gian chạy. Ví dụ:

```
interface Animal {
    name: string;
}
interface Dog extends Animal {
    bark: () => void;
}
interface Cat extends Animal {
    meow: () => void;
}
const makeNoise = (animal: Animal) => {
    if (animal instanceof Dog) {
        // 'Dog' chỉ đề cập đến một kiểu, nhưng đang được dùng làm giá trị ở đây.
        // ...
    }
};
```

Vì các kiểu bị xóa sau khi biên dịch, không có cách nào chạy đoạn mã này trong JavaScript. Để nhận biết kiểu tại thời gian chạy, chúng ta cần sử dụng một cơ chế khác. TypeScript cung cấp một số lựa chọn, trong đó cách phổ biến là "tagged union". Ví dụ:

```
interface Dog {
    kind: 'dog'; // Union có thể
    bark: () => void;
}
interface Cat {
    kind: 'cat'; // Union có thể
    meow: () => void;
}
type Animal = Dog | Cat;

const makeNoise = (animal: Animal) => {
    if (animal.kind === 'dog') {
        animal.bark();
    } else {
        animal.meow();
    }
};

const dog: Dog = {
    kind: 'dog',
    bark: () => console.log('bark'),
};
makeNoise(dog);
```

Thuộc tính "kind" là một giá trị có thể được sử dụng tại thời gian chạy để phân biệt các đối tượng trong JavaScript.

Một giá trị tại thời gian chạy cũng có thể có kiểu khác với kiểu được khai báo trong phần khai báo kiểu. Ví dụ, điều này có thể

xảy ra nếu lập trình viên hiểu sai kiểu của một API và chú thích nó không chính xác.

TypeScript là một tập cha của JavaScript, vì vậy từ khóa “class” có thể được sử dụng vừa như một kiểu vừa như một giá trị tại thời gian chạy.

```
class Animal {
    constructor(public name: string) {}
}
class Dog extends Animal {
    constructor(
        public name: string,
        public bark: () => void
    ) {
        super(name);
    }
}
class Cat extends Animal {
    constructor(
        public name: string,
        public meow: () => void
    ) {
        super(name);
    }
}
type Mammal = Dog | Cat;

const makeNoise = (mammal: Mammal) => {
    if (mammal instanceof Dog) {
        mammal.bark();
    } else {
        mammal.meow();
    }
};

const dog = new Dog('Fido', () => console.log('bark'));
makeNoise(dog);
```

Trong JavaScript, một “class” có thuộc tính “prototype”, và toán tử “instanceof” có thể được dùng để kiểm tra xem thuộc tính prototype của một constructor có xuất hiện ở bất kỳ đâu trong chuỗi prototype của một đối tượng hay không.

TypeScript không ảnh hưởng đến hiệu năng thời gian chạy vì mọi kiểu đều bị xóa. Tuy nhiên, TypeScript có tạo thêm một phần chi phí ở thời gian build.

JavaScript hiện đại ngay bây giờ (Downleveling)

TypeScript có thể biên dịch mã sang bất kỳ phiên bản JavaScript đã phát hành nào kể từ ECMAScript 3 (1999). Điều này có nghĩa TypeScript có thể chuyển dịch mã sử dụng các tính năng JavaScript mới nhất sang các phiên bản cũ hơn, một quá trình được gọi là Downleveling. Nhờ đó, có thể sử dụng JavaScript hiện đại trong khi vẫn duy trì mức tương thích tối đa với các môi trường thực thi cũ hơn.

Điều quan trọng cần lưu ý là khi chuyển dịch sang một phiên bản JavaScript cũ hơn, TypeScript có thể tạo ra mã phát sinh chi phí hiệu năng so với các triển khai native.

Dưới đây là một số tính năng JavaScript hiện đại có thể sử dụng trong TypeScript:

- ECMAScript modules thay cho các callback “define” kiểu AMD hoặc các câu lệnh “require” của CommonJS.
- Class thay cho prototype.
- Khai báo biến bằng “let” hoặc “const” thay cho “var”.
- Vòng lặp “for-of” hoặc “.forEach” thay cho vòng lặp “for” truyền thống.
- Arrow function thay cho function expression.

- Destructuring assignment.
- Tên thuộc tính/phương thức rút gọn và tên thuộc tính được tính toán.
- Tham số hàm mặc định.

Bằng cách tận dụng các tính năng JavaScript hiện đại này, lập trình viên có thể viết mã TypeScript biểu đạt rõ ràng và ngắn gọn hơn.

Bắt đầu với TypeScript

Cài đặt

Visual Studio Code hỗ trợ rất tốt ngôn ngữ TypeScript nhưng không bao gồm trình biên dịch TypeScript. Để cài đặt trình biên dịch TypeScript, bạn có thể sử dụng trình quản lý gói như npm hoặc yarn:

```
npm install typescript --save-dev
```

hoặc

```
yarn add typescript --dev
```

Hãy bảo đảm lockfile đã tạo được commit để mọi thành viên trong nhóm đều sử dụng cùng một phiên bản TypeScript.

Để chạy trình biên dịch TypeScript, bạn có thể sử dụng các lệnh sau:

```
npx tsc
```

hoặc

```
yarn tsc
```

Nên cài đặt TypeScript theo từng dự án thay vì cài đặt toàn cục vì cách này tạo ra quy trình build dễ dự đoán hơn. Tuy nhiên, với nhu cầu sử dụng một lần, bạn có thể dùng lệnh sau:

```
npx tsc
```

hoặc cài đặt toàn cục:

```
npm install -g typescript
```

Nếu đang sử dụng Microsoft Visual Studio, bạn có thể lấy TypeScript dưới dạng một package trong NuGet cho các dự án MSBuild. Trong NuGet Package Manager Console, chạy lệnh sau:

```
Install-Package Microsoft.TypeScript.MSBuild
```

Trong quá trình cài đặt TypeScript, hai tệp thực thi được cài đặt: “tsc” là trình biên dịch TypeScript và “tsserver” là máy chủ TypeScript độc lập. Máy chủ độc lập chứa trình biên dịch và các dịch vụ ngôn ngữ mà editor và IDE có thể sử dụng để cung cấp khả năng hoàn thành mã thông minh.

Ngoài ra, có một số trình chuyển dịch tương thích với TypeScript như Babel (thông qua plugin) hoặc swc. Các trình chuyển dịch này có thể được dùng để chuyển đổi mã TypeScript sang các ngôn ngữ hoặc phiên bản đích khác.

TypeScript 7.0 đã được viết lại bằng Go như một triển khai native của trình biên dịch và dịch vụ ngôn ngữ. Nó sử dụng đa luồng với bộ nhớ dùng chung cùng các tối ưu hóa khác để tăng tốc quá trình build đầy đủ và các tính năng của editor, giúp rút ngắn thời gian phản hồi trong quá trình phát triển.

Một số tính năng hiệu năng của TypeScript 7.0 có thể được tinh chỉnh. Việc kiểm tra kiểu có thể chạy trên các worker song song bằng `--checkers`; nhiều worker hơn có thể tăng tốc các dự án lớn nhưng sẽ dùng nhiều bộ nhớ hơn. Chế độ `--watch` được xây dựng lại cải thiện khả năng theo dõi tệp trên nhiều nền tảng. TypeScript 7.0 chưa bao gồm compiler API (tính đến tháng 7 năm 2026), vì vậy các công cụ vẫn cần API TypeScript 6.0 có thể chạy song song với TypeScript 7.0 bằng cách sử dụng `@typescript/typescript6` hoặc npm alias.

Cấu hình

Có thể cấu hình TypeScript bằng các tùy chọn CLI của tsc hoặc bằng một tệp cấu hình chuyên dụng có tên tsconfig.json được đặt ở thư mục gốc của dự án.

Để tạo tệp tsconfig.json được điền sẵn các thiết lập được khuyến nghị, bạn có thể dùng lệnh sau:

```
tsc --init
```

Khi thực thi lệnh `tsc` cục bộ, TypeScript sẽ biên dịch mã bằng cấu hình được chỉ định trong tệp tsconfig.json gần nhất.

Dưới đây là một số ví dụ về lệnh CLI chạy với thiết lập mặc định:

```
tsc main.ts // Biên dịch một tệp cụ thể (main.ts) sang JavaScript
tsc src/*.ts // Biên dịch mọi tệp .ts trong thư mục 'src' sang JavaScript
tsc app.ts util.ts --outfile index.js // Biên dịch hai tệp TypeScript (app.ts và util.ts) thành một tệp JavaScript duy nhất (index.js)
```

Tệp cấu hình TypeScript

Tệp tsconfig.json được dùng để cấu hình TypeScript Compiler (tsc). Thông thường, nó được thêm vào thư mục gốc của dự án cùng với tệp `package.json`.

Ghi chú:

- tsconfig.json chấp nhận comment dù có định dạng JSON.
- Nên sử dụng tệp cấu hình này thay cho các tùy chọn dòng lệnh.

Tại các liên kết sau, bạn có thể tìm thấy tài liệu đầy đủ và schema của nó:

https://www.typescriptlang.org/tsconfig

https://www.typescriptlang.org/tsconfig/

Sau đây là danh sách các cấu hình phổ biến và hữu ích:

target

Thuộc tính “target” được dùng để chỉ định phiên bản ECMAScript mà mã TypeScript của bạn sẽ được emit/biên dịch sang. Với các trình duyệt hiện đại, ES6 là một lựa chọn phù hợp. Lưu ý: hỗ trợ ES5 đã bị deprecated trong TypeScript 6.0 và không còn được hỗ trợ trong TypeScript 7.0.

lib

Thuộc tính “lib” được dùng để chỉ định các tệp thư viện cần đưa vào tại thời điểm biên dịch. TypeScript tự động bao gồm các API

cho những tính năng được chỉ định trong thuộc tính “target”, nhưng có thể bỏ hoặc chọn các thư viện cụ thể theo nhu cầu. Ví dụ, nếu đang làm việc với dự án máy chủ, bạn có thể loại trừ thư viện “DOM”, vốn chỉ hữu ích trong môi trường trình duyệt.

strict

Tùy chọn “strict” cải thiện độ an toàn kiểu bằng cách bật các kiểm tra chặt chẽ hơn. Tùy chọn này được bật mặc định kể từ TypeScript 6.0; nếu không, bạn nên đặt tường minh thành true trong tsconfig.json. Việc bật “strict” cho phép TypeScript:

- Emit mã bằng “use strict” cho từng tệp nguồn.
- Xem xét “null” và “undefined” trong quá trình kiểm tra kiểu.
- Vô hiệu hóa việc sử dụng kiểu “any” khi không có chú thích kiểu.
- Báo lỗi khi sử dụng biểu thức “this” mà nếu không sẽ ngầm mang kiểu “any”.

module

Thuộc tính “module” đặt hệ thống module được hỗ trợ cho chương trình đã biên dịch. Tại thời gian chạy, một module loader được dùng để định vị và thực thi các dependency dựa trên hệ thống module đã chỉ định.

Các module loader phổ biến nhất trong JavaScript là Node.js CommonJS cho ứng dụng phía máy chủ và RequireJS cho module AMD trong ứng dụng web chạy trên trình duyệt. TypeScript có thể emit mã cho nhiều hệ thống module khác nhau, bao gồm UMD, System, ESNext, ES2015/ES6 và ES2020. Hệ

thống module nên được chọn dựa trên môi trường đích và cơ chế tải module có sẵn trong môi trường đó.

Lưu ý: hỗ trợ các hệ thống module cũ hơn (AMD, UMD, SystemJS) đã bị deprecated trong TypeScript 6.0 và không còn được hỗ trợ trong TypeScript 7.0.

moduleResolution

Thuộc tính “moduleResolution” chỉ định chiến lược phân giải module. Sử dụng “nodenext” hoặc “bundler” cho mã TypeScript hiện đại. Chiến lược “classic” chỉ được dùng cho các phiên bản TypeScript cũ (trước 1.6).

esModuleInterop

Thuộc tính “esModuleInterop” cho phép default import từ các module CommonJS không export bằng thuộc tính “default”; thuộc tính này cung cấp một shim để bảo đảm khả năng tương thích trong JavaScript được emit. Sau khi bật tùy chọn này, chúng ta có thể dùng `import MyLibrary from "my-library"` thay cho `import * as MyLibrary from "my-library"`.

“esModuleInterop” ban đầu là tùy chọn opt-in để tránh breaking change, nhưng từ lâu đã là mặc định được khuyến nghị. Việc vô hiệu hóa nó có thể gây ra các vấn đề tinh vi tại thời gian chạy khi dùng CommonJS với ESM. Lưu ý: kể từ TypeScript 6.0, hành vi interop an toàn hơn này luôn được bật.

Trong TypeScript 6.0, một số tùy chọn cấu hình và dạng cú pháp cũ đã bị deprecated hoặc chuyển qua hành vi cũ. Trong

TypeScript 7.0, chúng trở thành lỗi nghiêm trọng hoặc hành vi no-op.

Các mục deprecated đã trở thành lỗi nghiêm trọng với hành vi no-op gồm:

- `target: es5`
- `downlevelIteration`
- `moduleResolution: node/node10`
- `module: amd/umd/systemjs/none`
- `baseUrl`
- `moduleResolution: classic`
- vô hiệu hóa `esModuleInterop` hoặc `allowSyntheticDefaultImports`
- vô hiệu hóa `alwaysStrict`
- từ khóa `module` trong khai báo namespace
- `asserts` trên import
- `/// <reference no-default-lib />` trong `skipDefaultLibCheck`
- đường dẫn tệp CLI với `tsconfig.json` cục bộ trừ khi sử dụng `--ignoreConfig`

jsx

Thuộc tính “jsx” chỉ áp dụng cho các tệp .tsx được dùng trong ReactJS và kiểm soát cách các cấu trúc JSX được biên dịch thành JavaScript. Một tùy chọn phổ biến là “preserve”, sẽ biên dịch thành tệp .jsx nhưng giữ nguyên JSX để có thể chuyển tiếp sang các công cụ khác như Babel cho các bước biến đổi tiếp theo.

skipLibCheck

Thuộc tính “skipLibCheck” ngăn TypeScript kiểm tra kiểu toàn bộ các package bên thứ ba đã import. Thuộc tính này giúp giảm thời gian biên dịch của dự án. TypeScript vẫn kiểm tra mã của bạn dựa trên các định nghĩa kiểu do các package này cung cấp.

files

Thuộc tính “files” cho trình biên dịch biết danh sách các tệp luôn phải được đưa vào chương trình.

include

Thuộc tính “include” cho trình biên dịch biết danh sách các tệp mà chúng ta muốn đưa vào. Thuộc tính này cho phép các pattern giống glob, chẳng hạn như “*” *cho mọi thư mục con,* “” cho mọi tên tệp và “?” cho các ký tự tùy chọn.

exclude

Thuộc tính “exclude” cho trình biên dịch biết danh sách các tệp không nên được đưa vào quá trình biên dịch. Danh sách này có thể bao gồm các tệp như “node_modules” hoặc các tệp kiểm thử. Lưu ý: tsconfig.json cho phép comment.

importHelpers

TypeScript sử dụng mã helper khi sinh mã cho một số tính năng JavaScript nâng cao hoặc được down-level. Theo mặc định, các helper này được lặp lại trong các tệp sử dụng chúng. Tùy chọn `importHelpers` thay vào đó import các helper từ module `tslib`, giúp đầu ra JavaScript hiệu quả hơn.

Lời khuyên khi chuyển sang TypeScript

Với các dự án lớn, nên áp dụng quá trình chuyển đổi dần dần, trong đó mã TypeScript và JavaScript ban đầu cùng tồn tại. Chỉ các dự án nhỏ mới có thể chuyển sang TypeScript trong một lần.

Bước đầu tiên của quá trình chuyển đổi này là đưa TypeScript vào chuỗi build. Có thể thực hiện bằng tùy chọn trình biên dịch "allowJs", cho phép các tệp .ts và .tsx cùng tồn tại với các tệp JavaScript hiện có. Vì TypeScript sẽ fallback sang kiểu "any" cho một biến khi không thể suy luận kiểu từ các tệp JavaScript, nên ở giai đoạn đầu của quá trình chuyển đổi, nên vô hiệu hóa "noImplicitAny" trong các tùy chọn trình biên dịch.

Bước thứ hai là bảo đảm các bài kiểm thử JavaScript hoạt động cùng các tệp TypeScript để bạn có thể chạy kiểm thử khi chuyển đổi từng module. Nếu đang dùng Jest, hãy cân nhắc sử dụng `ts-jest`, cho phép kiểm thử các dự án TypeScript bằng Jest.

Bước thứ ba là thêm các khai báo kiểu cho thư viện bên thứ ba vào dự án. Các khai báo này có thể được đóng gói cùng thư viện hoặc có trên DefinitelyTyped. Bạn có thể tìm chúng tại https://www.typescriptlang.org/dt/search và cài đặt bằng:

```
npm install --save-dev @types/package-name
```

hoặc

```
yarn add --dev @types/package-name
```

Bước thứ tư là chuyển đổi từng module theo cách tiếp cận từ dưới lên, dựa trên Dependency Graph và bắt đầu từ các node lá. Ý tưởng là bắt đầu chuyển đổi những Module không phụ thuộc

vào Module khác. Để trực quan hóa các dependency graph, bạn có thể dùng công cụ “madge”.

Các module phù hợp cho những bước chuyển đổi ban đầu này là các hàm tiện ích và mã liên quan đến API bên ngoài hoặc specification. Có thể tự động tạo các định nghĩa kiểu TypeScript từ hợp đồng Swagger, GraphQL hoặc JSON schema để đưa vào dự án.

Khi không có specification hoặc schema chính thức, bạn có thể tạo kiểu từ dữ liệu thô, chẳng hạn JSON do máy chủ trả về. Tuy nhiên, nên tạo kiểu từ specification thay vì từ dữ liệu để tránh bỏ sót các trường hợp biên.

Trong quá trình chuyển đổi, không nên refactor mã và chỉ tập trung vào việc thêm kiểu cho các module.

Bước thứ năm là bật “noImplicitAny”, điều này sẽ bắt buộc mọi kiểu phải được biết và định nghĩa, mang lại trải nghiệm TypeScript tốt hơn cho dự án.

Trong quá trình chuyển đổi, bạn có thể dùng directive `@ts-check`, cho phép TypeScript kiểm tra kiểu trong tệp JavaScript. Directive này cung cấp một phiên bản kiểm tra kiểu lỏng hơn và ban đầu có thể dùng để xác định vấn đề trong các tệp JavaScript. Khi `@ts-check` được thêm vào tệp, TypeScript sẽ cố gắng suy ra các định nghĩa bằng comment theo phong cách JSDoc. Tuy nhiên, chỉ nên cân nhắc dùng chú thích JSDoc ở giai đoạn rất sớm của quá trình chuyển đổi.

Hãy cân nhắc giữ giá trị mặc định của `noEmitOnError` trong tsconfig.json là false. Điều này cho phép bạn xuất mã nguồn JavaScript ngay cả khi có lỗi được báo cáo.

Khám phá hệ thống kiểu

Dịch vụ ngôn ngữ TypeScript

Dịch vụ ngôn ngữ TypeScript, còn được gọi là tsserver, cung cấp nhiều tính năng như báo lỗi, chẩn đoán, biên dịch khi lưu, đổi tên, đi đến định nghĩa, danh sách gợi ý hoàn thành, hỗ trợ chữ ký và nhiều tính năng khác. Dịch vụ này chủ yếu được các môi trường phát triển tích hợp (IDE) sử dụng để cung cấp hỗ trợ IntelliSense. Nó tích hợp liền mạch với Visual Studio Code và được các công cụ như Conquer of Completion (Coc) sử dụng.

Lập trình viên có thể tận dụng một API chuyên dụng và tạo các plugin dịch vụ ngôn ngữ tùy chỉnh của riêng mình để cải thiện trải nghiệm chỉnh sửa TypeScript. Điều này có thể đặc biệt hữu ích để triển khai các tính năng lint đặc thù hoặc bật tự động hoàn thành cho một ngôn ngữ template tùy chỉnh.

Một ví dụ thực tế về plugin tùy chỉnh là “typescript-styled-plugin”, cung cấp khả năng báo lỗi cú pháp và hỗ trợ IntelliSense cho các thuộc tính CSS trong styled components.

Để biết thêm thông tin và xem các hướng dẫn bắt đầu nhanh, bạn có thể tham khảo TypeScript Wiki chính thức trên GitHub: https://github.com/microsoft/TypeScript/wiki/

Kiểu cấu trúc

TypeScript dựa trên hệ thống kiểu cấu trúc. Điều này có nghĩa là tính tương thích và tương đương của các kiểu được xác định bởi cấu trúc hoặc định nghĩa thực tế của kiểu, thay vì tên hoặc nơi khai báo của nó như trong các hệ thống kiểu định danh như C# hoặc C.

Hệ thống kiểu cấu trúc của TypeScript được thiết kế dựa trên cách hệ thống duck typing động của JavaScript hoạt động tại thời gian chạy.

Ví dụ sau là mã TypeScript hợp lệ. Như bạn có thể thấy, “X” và “Y” có cùng thành viên “a”, mặc dù chúng có tên khai báo khác nhau. Các kiểu được xác định bởi cấu trúc của chúng và trong trường hợp này, vì các cấu trúc giống nhau nên chúng tương thích và hợp lệ.

```
type X = {
    a: string;
};
type Y = {
    a: string;
};
const x: X = { a: 'a' };
const y: Y = x; // Hợp lệ
```

Các quy tắc so sánh cơ bản của TypeScript

Quá trình so sánh của TypeScript có tính đệ quy và được thực hiện trên các kiểu lồng nhau ở bất kỳ cấp độ nào.

Một kiểu “X” tương thích với “Y” nếu “Y” có ít nhất các thành viên giống như “X”.

```
type X = {
    a: string;
};
const y = { a: 'A', b: 'B' }; // Hợp lệ, vì có ít nhất các thành viên giống X
const r: X = y;
```

Các tham số hàm được so sánh theo kiểu, không theo tên:

```
type X = (a: number) => void;
type Y = (a: number) => void;
let x: X = (j: number) => undefined;
let y: Y = (k: number) => undefined;
y = x; // Hợp lệ
x = y; // Hợp lệ
```

Kiểu trả về của hàm phải giống nhau:

```
type X = (a: number) => undefined;
type Y = (a: number) => number;
let x: X = (a: number) => undefined;
let y: Y = (a: number) => 1;
y = x; // Không hợp lệ
x = y; // Không hợp lệ
```

Kiểu trả về của hàm nguồn phải là kiểu con của kiểu trả về của hàm đích:

```
let x = () => ({ a: 'A' });
let y = () => ({ a: 'A', b: 'B' });
x = y; // Hợp lệ
y = x; // Không hợp lệ, thiếu thành viên b
```

Việc bỏ qua tham số hàm được cho phép vì đây là cách làm phổ biến trong JavaScript, chẳng hạn khi sử dụng “Array.prototype.map()”:

```
[1, 2, 3].map((element, _index, _array) => element + 'x');
```

Do đó, các khai báo kiểu sau hoàn toàn hợp lệ:

```
type X = (a: number) => undefined;
type Y = (a: number, b: number) => undefined;
let x: X = (a: number) => undefined;
let y: Y = (a: number) => undefined; // Thiếu tham số b
y = x; // Hợp lệ
```

Mọi tham số tùy chọn bổ sung của kiểu nguồn đều hợp lệ:

```
type X = (a: number, b?: number, c?: number) => undefined;
type Y = (a: number) => undefined;
let x: X = a => undefined;
let y: Y = a => undefined;
y = x; // Hợp lệ
x = y; //Hợp lệ
```

Mọi tham số tùy chọn của kiểu đích không có tham số tương ứng trong kiểu nguồn đều hợp lệ và không phải là lỗi:

```
type X = (a: number) => undefined;
type Y = (a: number, b?: number) => undefined;
let x: X = a => undefined;
let y: Y = a => undefined;
y = x; // Hợp lệ
x = y; // Hợp lệ
```

Rest parameter được coi như một chuỗi vô hạn các tham số tùy chọn:

```
type X = (a: number, ...rest: number[]) => undefined;
let x: X = a => undefined; //hợp lệ
```

Các hàm có overload hợp lệ nếu chữ ký overload tương thích với chữ ký triển khai:

```
function x(a: string): void;
function x(a: string, b: number): void;
function x(a: string, b?: number): void {
    console.log(a, b);
}
x('a'); // Hợp lệ
x('a', 1); // Hợp lệ

function y(a: string): void; // Không hợp lệ, không tương thích
với chữ ký triển khai
```

```
function y(a: string, b: number): void;
function y(a: string, b: number): void {
    console.log(a, b);
}
y('a');
y('a', 1);
```

Việc so sánh tham số hàm thành công nếu các tham số nguồn và đích có thể gán cho kiểu cha hoặc kiểu con (bivariance).

```
// Kiểu cha
class X {
    a: string;
    constructor(value: string) {
        this.a = value;
    }
}
// Kiểu con
class Y extends X {}
// Kiểu con
class Z extends X {}

type GetA = (x: X) => string;
const getA: GetA = x => x.a;

// Tính song biến chấp nhận các kiểu cha
console.log(getA(new X('x'))); // Hợp lệ
console.log(getA(new Y('Y'))); // Hợp lệ
console.log(getA(new Z('z'))); // Hợp lệ
```

Enum có thể so sánh và tương thích với số và ngược lại, nhưng việc so sánh các giá trị Enum từ các kiểu Enum khác nhau là không hợp lệ.

```
enum X {
    A,
    B,
}
```

```
enum Y {
    A,
    B,
    C,
}
const xa: number = X.A; // Hợp lệ
const ya: Y = 0; // Hợp lệ
X.A === Y.A; // Không hợp lệ
```

Các instance của một class phải trải qua kiểm tra tương thích đối với các thành viên private và protected của chúng:

```
class X {
    public a: string;
    constructor(value: string) {
        this.a = value;
    }
}

class Y {
    private a: string;
    constructor(value: string) {
        this.a = value;
    }
}

let x: X = new Y('y'); // Không hợp lệ
```

Kiểm tra so sánh không xét đến các cây phân cấp kế thừa khác nhau, ví dụ:

```
class X {
    public a: string;
    constructor(value: string) {
        this.a = value;
    }
}
class Y extends X {
```

```
    public a: string;
    constructor(value: string) {
        super(value);
        this.a = value;
    }
}
class Z {
    public a: string;
    constructor(value: string) {
        this.a = value;
    }
}
let x: X = new X('x');
let y: Y = new Y('y');
let z: Z = new Z('z');
x === y; // Hợp lệ
x === z; // Hợp lệ dù z thuộc một hệ thống phân cấp kế thừa khác
```

Generic được so sánh bằng cấu trúc dựa trên kiểu kết quả sau khi áp dụng tham số generic; chỉ kết quả cuối cùng được so sánh như một kiểu không generic.

```
interface X<T> {
    a: T;
}
let x: X<number> = { a: 1 };
let y: X<string> = { a: 'a' };
x === y; // Không hợp lệ vì đối số kiểu được dùng trong cấu trúc cuối cùng

interface X<T> {}
const x: X<number> = 1;
const y: X<string> = 'a';
x === y; // Hợp lệ vì đối số kiểu không được dùng trong cấu trúc cuối cùng
```

Khi generic không chỉ định type argument, tất cả argument chưa được chỉ định được xem là kiểu “any”:

```
type X = <T>(x: T) => T;
type Y = <K>(y: K) => K;
let x: X = x => x;
let y: Y = y => y;
x = y; // Hợp lệ
```

Hãy nhớ:

```
let a: number = 1;
let b: number = 2;
a = b; // Hợp lệ, mọi kiểu đều có thể gán cho chính nó

let c: any;
c = 1; // Hợp lệ, mọi kiểu đều có thể gán cho any

let d: unknown;
d = 1; // Hợp lệ, mọi kiểu đều có thể gán cho unknown

let e: unknown;
let e1: unknown = e; // Hợp lệ, unknown chỉ có thể gán cho chính nó và any
let e2: any = e; // Hợp lệ
let e3: number = e; // Không hợp lệ

let f: never;
f = 1; // Không hợp lệ, không có gì có thể gán cho never

let g: void;
let g1: any;
g = 1; // Không hợp lệ, void không thể gán cho hoặc nhận giá trị từ bất cứ kiểu nào ngoại trừ any
g = g1; // Hợp lệ
```

Lưu ý rằng khi “strictNullChecks” được bật, “null” và “undefined” được xử lý tương tự “void”; nếu không, chúng

tương tự “never”.

Kiểu dưới dạng tập hợp

Trong TypeScript, một kiểu là một tập hợp các giá trị có thể có. Tập hợp này còn được gọi là miền của kiểu. Mỗi giá trị của một kiểu có thể được xem là một phần tử trong tập hợp. Một kiểu thiết lập các ràng buộc mà mọi phần tử trong tập hợp phải thỏa mãn để được coi là thành viên của tập hợp đó. Nhiệm vụ chính của TypeScript là kiểm tra và xác minh xem một tập hợp có phải là tập con của tập hợp khác hay không.

TypeScript hỗ trợ nhiều loại tập hợp:

Thuật ngữ tập hợp	**TypeScript**	**Ghi chú**
Tập rỗng	never	“never” chứa mọi thứ ngoài chính nó
Tập một phần tử	undefined / null / literal type	
Tập hữu hạn	boolean / union	
Tập vô hạn	string / number / object	
Tập phổ quát	any / unknown	Mọi phần tử là thành viên của “any” và mọi tập hợp là tập con của nó / “unknown” là đối ứng an toàn kiểu của “any”

Dưới đây là một vài ví dụ:

TypeScript	Thuật ngữ tập hợp	Ví dụ
never	∅ (tập rỗng)	const x: never = ‘x’; // Error: Type ‘string’ is not assignable to type ‘never’
Literal type	Tập một phần tử	type X = ‘X’; type Y = 7;
Value assignable to T	Giá trị ∈ T (thành viên)	type XY = ‘X’ | ‘Y’; const x: XY = ‘X’;
T1 assignable to T2	T1 ⊆ T2 (tập con của)	type XY = ‘X’ | ‘Y’; const x: XY = ‘X’; const j: XY = ‘J’; // Type ‘“J”’ is not assignable to type ‘XY’.
T1 extends T2	T1 ⊆ T2 (tập con của)	type X = ‘X’ extends string ? true : false;
T1 | T2	T1 ∪ T2 (hợp)	type XY = ‘X’ | ‘Y’; type JK = 1 | 2;
T1 & T2	T1 ∩ T2 (giao)	type X = { a: string } type Y = { b: string } type XY = X & Y const x: XY = { a: ‘a’, b: ‘b’ }
unknown	Tập phổ quát	const x: unknown = 1

Một union, (T1 | T2), tạo ra một tập hợp rộng hơn (cả hai):

```
type X = {
    a: string;
};
type Y = {
    b: string;
```

```
};
type XY = X | Y;
const r: XY = { a: 'a', b: 'x' }; // Hợp lệ
```

Một intersection, (T1 & T2), tạo ra một tập hợp hẹp hơn (chỉ phần chung):

```
type X = {
    a: string;
};
type Y = {
    a: string;
    b: string;
};
type XY = X & Y;
const r: XY = { a: 'a' }; // Không hợp lệ
const j: XY = { a: 'a', b: 'b' }; // Hợp lệ
```

Trong ngữ cảnh này, từ khóa `extends` có thể được xem là "tập con của". Nó đặt một ràng buộc cho một kiểu. Khi `extends` được sử dụng với generic, nó giới hạn tham số kiểu generic thành một kiểu cụ thể hơn.

Lưu ý rằng `extends` ở đây không liên quan đến kế thừa class theo nghĩa OOP.

TypeScript làm việc với các kiểu cấu trúc và không có một hệ phân cấp định danh nghiêm ngặt. Thực tế, như trong ví dụ dưới đây, hai kiểu có thể chồng lấp mà không kiểu nào là kiểu con của kiểu kia, vì TypeScript xem xét cấu trúc, hay hình dạng, của đối tượng.

```
interface X {
    a: string;
}
interface Y extends X {
```

```
    b: string;
}
interface Z extends Y {
    c: string;
}
const z: Z = { a: 'a', b: 'b', c: 'c' };
interface X1 {
    a: string;
}
interface Y1 {
    a: string;
    b: string;
}
interface Z1 {
    a: string;
    b: string;
    c: string;
}
const z1: Z1 = { a: 'a', b: 'b', c: 'c' };

const r: Z1 = z; // Hợp lệ
```

Gán kiểu: Khai báo kiểu và Type Assertion

Một kiểu có thể được gán theo nhiều cách khác nhau trong TypeScript:

Khai báo kiểu

Trong ví dụ sau, chúng ta sử dụng x: X (“: Type”) để khai báo kiểu cho biến x.

```
type X = {
    a: string;
};

// Khai báo kiểu
```

```
const x: X = {
    a: 'a',
};
```

Nếu biến không có định dạng đã chỉ định, TypeScript sẽ báo lỗi. Ví dụ:

```
type X = {
    a: string;
};

const x: X = {
    a: 'a',
     b: 'b', // Lỗi: Đối tượng literal chỉ được chỉ định các thuộc tính đã biết
};
```

Type Assertion

Có thể thêm một assertion bằng từ khóa `as`. Điều này cho trình biên dịch biết rằng lập trình viên có nhiều thông tin hơn về một kiểu và bỏ qua mọi lỗi có thể xảy ra.

Ví dụ:

```
type X = {
    a: string;
};
const x = {
    a: 'a',
    b: 'b',
} as X;
```

Trong ví dụ trên, đối tượng x được xác nhận là có kiểu X bằng từ khóa as. Điều này cho trình biên dịch TypeScript biết rằng

đối tượng tuân theo kiểu đã chỉ định, mặc dù nó có thêm thuộc tính b không có trong định nghĩa kiểu.

Type assertion hữu ích trong những tình huống cần chỉ định một kiểu cụ thể hơn, đặc biệt khi làm việc với DOM. Ví dụ:

```
const myInput = document.getElementById('my_input') as HTMLInputElement;
```

Ở đây, type assertion as HTMLInputElement được sử dụng để cho TypeScript biết kết quả của getElementById nên được xem như một HTMLInputElement. Type assertion cũng có thể được dùng để ánh xạ lại các key, như trong ví dụ dưới đây với template literal:

```
type J<Type> = {
    [
        Property in keyof Type as `prefix_${string & Property}`
    ]: () => Type[Property];
};
type X = {
    a: string;
    b: number;
};
type Y = J<X>;
```

Trong ví dụ này, kiểu `J<Type>` sử dụng mapped type với template literal để ánh xạ lại các key của `Type`. Nó tạo các thuộc tính mới với “prefix_” được thêm vào mỗi key và các giá trị tương ứng là các hàm trả về giá trị thuộc tính ban đầu.

Cần lưu ý rằng khi sử dụng type assertion, TypeScript sẽ không thực hiện excess property checking. Vì vậy, nói chung nên sử dụng Khai báo kiểu khi cấu trúc của đối tượng đã được biết trước.

Khai báo ambient

Khai báo ambient là các tệp mô tả kiểu cho mã JavaScript, chúng có định dạng tên tệp là `.d.ts`. Chúng thường được import và dùng để chú thích các thư viện JavaScript hiện có hoặc thêm kiểu vào các tệp JS hiện có trong dự án của bạn.

Có thể tìm thấy kiểu cho nhiều thư viện phổ biến tại: https://github.com/DefinitelyTyped/DefinitelyTyped/

và có thể cài đặt bằng:

```
npm install --save-dev @types/library-name
```

Với các Khai báo ambient do bạn định nghĩa, bạn có thể import bằng tham chiếu "triple-slash":

```
/// <reference path="./library-types.d.ts" />
```

Bạn có thể sử dụng Khai báo ambient ngay cả trong các tệp JavaScript bằng `// @ts-check`.

Từ khóa `declare` cho phép định nghĩa kiểu cho mã JavaScript hiện có mà không cần import nó, đóng vai trò như placeholder cho các kiểu từ tệp khác hoặc từ phạm vi global.

Kiểm tra thuộc tính và kiểm tra thuộc tính dư thừa

TypeScript dựa trên hệ thống kiểu cấu trúc nhưng excess property checking là một đặc tính của TypeScript cho phép kiểm tra liệu một đối tượng có đúng các thuộc tính được chỉ định trong kiểu hay không.

Excess Property Checking được thực hiện khi gán object literal cho biến hoặc khi truyền chúng làm đối số cho tham số của hàm, chẳng hạn.

```
type X = {
    a: string;
};
const y = { a: 'a', b: 'b' };
const x: X = y; // Hợp lệ nhờ hệ thống kiểu cấu trúc
const w: X = { a: 'a', b: 'b' }; // Không hợp lệ do kiểm tra thuộc tính dư thừa
```

Kiểu yếu

Một kiểu được coi là yếu khi nó không chứa gì ngoài một tập hợp toàn các thuộc tính tùy chọn:

```
type X = {
    a?: string;
    b?: string;
};
```

TypeScript coi việc gán bất kỳ thứ gì vào một kiểu yếu là lỗi khi không có phần chồng lấp; ví dụ, đoạn sau gây lỗi:

```
type Options = {
    a?: string;
    b?: string;
};

const fn = (options: Options) => undefined;

fn({ c: 'c' }); // Không hợp lệ
```

Mặc dù không được khuyến nghị, nếu cần, có thể bỏ qua kiểm tra này bằng type assertion:

```ts
type Options = {
    a?: string;
    b?: string;
};
const fn = (options: Options) => undefined;
fn({ c: 'c' } as Options); // Hợp lệ
```

Hoặc bằng cách thêm unknown vào index signature của kiểu yếu:

```ts
type Options = {
    [prop: string]: unknown;
    a?: string;
    b?: string;
};

const fn = (options: Options) => undefined;
fn({ c: 'c' }); // Hợp lệ
```

Kiểm tra nghiêm ngặt object literal (Freshness)

Kiểm tra nghiêm ngặt object literal, đôi khi được gọi là "freshness", là một tính năng trong TypeScript giúp phát hiện các thuộc tính dư thừa hoặc viết sai chính tả mà nếu không sẽ không bị nhận ra trong các kiểm tra kiểu cấu trúc thông thường.

Khi tạo một object literal, trình biên dịch TypeScript xem nó là "fresh". Nếu object literal được gán cho một biến hoặc truyền làm tham số, TypeScript sẽ báo lỗi nếu object literal chỉ định các thuộc tính không tồn tại trong kiểu đích.

Tuy nhiên, "freshness" biến mất khi object literal được widening hoặc khi sử dụng type assertion.

Dưới đây là một số ví dụ minh họa:

```ts
type X = { a: string };
type Y = { a: string; b: string };

let x: X;
x = { a: 'a', b: 'b' }; // Kiểm tra freshness: Phép gán không hợp lệ
var y: Y;
y = { a: 'a', bx: 'bx' }; // Kiểm tra freshness: Phép gán không hợp lệ

const fn = (x: X) => console.log(x.a);

fn(x);
fn(y); // Mở rộng kiểu: Không có lỗi, kiểu tương thích về cấu trúc

fn({ a: 'a', bx: 'b' }); // Kiểm tra freshness: Đối số không hợp lệ

let c: X = { a: 'a' };
let d: Y = { a: 'a', b: '' };
c = d; // Mở rộng kiểu: Không kiểm tra freshness
```

Suy luận kiểu

TypeScript có thể suy luận kiểu khi không có chú thích được cung cấp trong:

- Khởi tạo biến.
- Khởi tạo thành viên.
- Đặt giá trị mặc định cho tham số.
- Kiểu trả về của hàm.

Ví dụ:

```ts
let x = 'x'; // Kiểu được suy luận là string
```

Trình biên dịch TypeScript phân tích giá trị hoặc biểu thức và xác định kiểu của nó dựa trên thông tin có sẵn.

Suy luận nâng cao hơn

Khi nhiều biểu thức được sử dụng trong suy luận kiểu, TypeScript tìm kiếm "kiểu chung tốt nhất". Ví dụ:

```
let x = [1, 'x', 1, null]; // Kiểu được suy luận là: (string | number | null)[]
```

Nếu trình biên dịch không thể tìm thấy kiểu chung tốt nhất, nó trả về một union type. Ví dụ:

```
let x = [new RegExp('x'), new Date()]; // Kiểu được suy luận là: (RegExp | Date)[]
```

TypeScript sử dụng "contextual typing" dựa trên vị trí của biến để suy luận kiểu. Trong ví dụ sau, trình biên dịch biết rằng `e` có kiểu `MouseEvent` nhờ kiểu sự kiện `click` được định nghĩa trong tệp lib.d.ts, chứa các khai báo ambient cho nhiều cấu trúc JavaScript phổ biến và DOM:

```
window.addEventListener('click', function (e) {}); // Kiểu được suy luận của e là MouseEvent
```

Type Widening

Type widening là quá trình TypeScript gán một kiểu cho biến được khởi tạo mà không có chú thích kiểu. Nó cho phép chuyển từ kiểu hẹp sang kiểu rộng hơn nhưng không theo chiều ngược lại. Trong ví dụ sau:

```
let x = 'x'; // TypeScript suy luận thành string, một kiểu rộng
let y: 'y' | 'x' = 'y'; // kiểu của y là union của các kiểu literal
```

```
y = x; // Không hợp lệ, kiểu 'string' không thể gán cho kiểu
'"x" | "y"'.
```

TypeScript gán `string` cho `x` dựa trên giá trị duy nhất được cung cấp khi khởi tạo (`x`); đây là một ví dụ về widening.

TypeScript cung cấp các cách để kiểm soát quá trình widening, chẳng hạn sử dụng “const”.

Const

Sử dụng từ khóa `const` khi khai báo biến dẫn đến suy luận kiểu hẹp hơn trong TypeScript.

Ví dụ:

```
const x = 'x'; // TypeScript suy luận kiểu của x là 'x', một
kiểu hẹp hơn
let y: 'y' | 'x' = 'y';
y = x; // Hợp lệ: Kiểu của x được suy luận là 'x'
```

Bằng cách dùng `const` để khai báo biến x, kiểu của nó được thu hẹp thành giá trị literal cụ thể ‘x’. Vì kiểu của x được thu hẹp, nó có thể được gán cho biến y mà không có lỗi. Lý do kiểu có thể được suy luận như vậy là vì biến `const` không thể được gán lại, nên kiểu của nó có thể được thu hẹp xuống một literal type cụ thể, trong trường hợp này là literal type ‘x’.

Modifier Const trên tham số kiểu

Từ TypeScript phiên bản 5.0, có thể chỉ định thuộc tính `const` trên một tham số kiểu generic. Điều này cho phép suy luận kiểu chính xác nhất có thể. Hãy xem một ví dụ không sử dụng `const`:

```
function identity<T>(value: T) {
    // Không có const ở đây
    return value;
}
const values = identity({ a: 'a', b: 'b' }); // Kiểu được suy
luận là: { a: string; b: string; }
```

Như bạn có thể thấy, các thuộc tính `a` và `b` được suy luận có kiểu `string`.

Bây giờ, hãy xem sự khác biệt với phiên bản `const`:

```
function identity<const T>(value: T) {
    // Dùng từ khóa bổ nghĩa const trên tham số kiểu
    return value;
}
const values = identity({ a: 'a', b: 'b' }); // Kiểu được suy
luận là: { a: "a"; b: "b"; }
```

Bây giờ chúng ta có thể thấy các thuộc tính `a` và `b` được suy luận là các string literal thay vì chỉ là kiểu `string`.

Const assertion

Tính năng này cho phép bạn khai báo một biến với literal type chính xác hơn dựa trên giá trị khởi tạo của nó, cho trình biên dịch biết rằng giá trị nên được xem như một literal bất biến. Dưới đây là một vài ví dụ:

Trên một thuộc tính:

```
const v = {
    x: 3 as const,
};
v.x = 3;
```

Trên toàn bộ đối tượng:

```
const v = {
    x: 1,
    y: 2,
} as const;
```

Điều này có thể đặc biệt hữu ích khi định nghĩa kiểu cho tuple:

```
const x = [1, 2, 3]; // number[]
const y = [1, 2, 3] as const; // Tuple readonly [1, 2, 3]
```

Chú thích kiểu tường minh

Chúng ta có thể chỉ định cụ thể và truyền một kiểu. Trong ví dụ sau, thuộc tính `x` có kiểu `number`:

```
const v = {
    x: 1, // Kiểu được suy luận: number (mở rộng kiểu)
};
v.x = 3; // Hợp lệ
```

Chúng ta có thể làm cho chú thích kiểu cụ thể hơn bằng cách sử dụng một union của các literal type:

```
const v: { x: 1 | 2 | 3 } = {
    x: 1, // x hiện là union của các kiểu literal: 1 | 2 | 3
};
v.x = 3; // Hợp lệ
v.x = 100; // Không hợp lệ
```

Thu hẹp kiểu

Type Narrowing là quá trình trong TypeScript mà một kiểu tổng quát được thu hẹp xuống một kiểu cụ thể hơn. Điều này xảy ra khi TypeScript phân tích mã và xác định rằng một số điều kiện hoặc thao tác có thể tinh chỉnh thông tin kiểu.

Việc thu hẹp kiểu có thể xảy ra theo nhiều cách, bao gồm:

Điều kiện

Bằng cách sử dụng các câu lệnh điều kiện như `if` hoặc `switch`, TypeScript có thể thu hẹp kiểu dựa trên kết quả của điều kiện. Ví dụ:

```
let x: number | undefined = 10;

if (x !== undefined) {
    x += 100; // Kiểu là number, đã được thu hẹp bởi điều kiện
}
```

Throw hoặc return

Việc throw một lỗi hoặc return sớm khỏi một nhánh có thể được dùng để giúp TypeScript thu hẹp kiểu. Ví dụ:

```
let x: number | undefined = 10;

if (x === undefined) {
    throw 'error';
}
x += 100;
```

Các cách khác để thu hẹp kiểu trong TypeScript bao gồm:

- Toán tử `instanceof`: Dùng để kiểm tra một đối tượng có phải là instance của một class cụ thể hay không.
- Toán tử `in`: Dùng để kiểm tra một thuộc tính có tồn tại trong một đối tượng hay không.
- Toán tử `typeof`: Dùng để kiểm tra kiểu của một giá trị tại thời gian chạy.
- Các hàm tích hợp như `Array.isArray()`: Dùng để kiểm tra một giá trị có phải là mảng hay không.

Discriminated Union

Sử dụng "Discriminated Union" là một pattern trong TypeScript, trong đó một "tag" tường minh được thêm vào các đối tượng để phân biệt các kiểu khác nhau trong một union. Pattern này còn được gọi là "tagged union". Trong ví dụ sau, "tag" được biểu diễn bởi thuộc tính "type":

```
type A = { type: 'type_a'; value: number };
type B = { type: 'type_b'; value: string };

const x = (input: A | B): string | number => {
    switch (input.type) {
        case 'type_a':
            return input.value + 100; // kiểu là A
        case 'type_b':
            return input.value + 'extra'; // kiểu là B
    }
};
```

Type Guard do người dùng định nghĩa

Trong những trường hợp TypeScript không thể xác định một kiểu, có thể viết một hàm trợ giúp được gọi là "user-defined type guard". Trong ví dụ sau, chúng ta sẽ sử dụng Type Predicate để thu hẹp kiểu sau khi áp dụng một số bước lọc:

```
const data = ['a', null, 'c', 'd', null, 'f'];

const r1 = data.filter(x => x != null); // Kiểu là (string | null)[], TypeScript không thể suy luận kiểu chính xác

const isValid = (item: string | null): item is string => item !== null; // Bộ kiểm tra kiểu tùy chỉnh
```

```
const r2 = data.filter(isValid); // Kiểu hiện đã đúng là string[], nhờ dùng bộ kiểm tra kiểu vị từ để thu hẹp kiểu
```

Thu hẹp bằng switch-true

TypeScript 5.3 bổ sung switch-true narrowing, cho phép bạn thay các chuỗi if/else rối rắm bằng switch (true) sử dụng các điều kiện boolean. Cách này cải thiện khả năng đọc mà vẫn thu hẹp kiểu. Nó tương tự pattern matching nhưng đơn giản hơn.

```
function classify(x: unknown) {
    switch (true) {
        case typeof x === 'string':
            return `"${x.toUpperCase()}"`;
        case typeof x === 'number':
            return x > 0 ? 'positive' : 'negative';
        case Array.isArray(x):
            return `[${x.length} items]`;
        default:
            return 'something else';
    }
}
```

Các kiểu nguyên thủy

TypeScript hỗ trợ 7 kiểu nguyên thủy. Kiểu dữ liệu nguyên thủy là kiểu không phải đối tượng và không có phương thức nào gắn với nó. Trong TypeScript, mọi kiểu nguyên thủy đều bất biến, nghĩa là giá trị của chúng không thể thay đổi sau khi được gán.

string

Kiểu nguyên thủy `string` lưu trữ dữ liệu văn bản và giá trị luôn được đặt trong dấu nháy kép hoặc nháy đơn.

```
const x: string = 'x';
const y: string = 'y';
```

Chuỗi có thể trải dài trên nhiều dòng nếu được bao quanh bởi ký tự backtick (`):

```
let sentence: string = `xxx,
   yyy`;
```

boolean

Kiểu dữ liệu `boolean` trong TypeScript lưu một giá trị nhị phân, hoặc `true` hoặc `false`.

```
const isReady: boolean = true;
```

number

Kiểu dữ liệu `number` trong TypeScript được biểu diễn bằng giá trị dấu phẩy động 64-bit. Kiểu `number` có thể biểu diễn số nguyên và số thập phân. TypeScript cũng hỗ trợ hệ thập lục phân, nhị phân và bát phân, ví dụ:

```
const decimal: number = 10;
const hexadecimal: number = 0xa00d; // Số thập lục phân bắt đầu
bằng 0x
const binary: number = 0b1010; // Số nhị phân bắt đầu bằng 0b
const octal: number = 0o633; // Số bát phân bắt đầu bằng 0o
```

bigint

`bigint` biểu diễn các giá trị số nguyên có thể lớn hơn số nguyên an toàn tối đa được `number` hỗ trợ, là 2^53 - 1.

Có thể tạo `bigint` bằng cách gọi hàm tích hợp `BigInt()` hoặc thêm `n` vào cuối bất kỳ literal số nguyên nào:

```
const x: bigint = BigInt(9007199254740991);
const y: bigint = 9007199254740991n;
```

Ghi chú:

- Giá trị `bigint` không thể trộn với `number` và không thể sử dụng với `Math` tích hợp; chúng phải được ép về cùng kiểu.
- Giá trị `bigint` chỉ khả dụng nếu cấu hình target là ES2020 trở lên.

Symbol

Symbol là các định danh duy nhất có thể được sử dụng làm key thuộc tính trong đối tượng để ngăn xung đột tên.

```
type Obj = {
    [sym: symbol]: number;
};

const a = Symbol('a');
const b = Symbol('b');
let obj: Obj = {};
obj[a] = 123;
obj[b] = 456;

console.log(obj[a]); // 123
console.log(obj[b]); // 456
```

null và undefined

Các kiểu `null` và `undefined` đều biểu diễn không có giá trị hoặc sự vắng mặt của bất kỳ giá trị nào.

Kiểu `undefined` có nghĩa giá trị chưa được gán hoặc khởi tạo, hoặc biểu thị sự vắng mặt ngoài ý muốn của một giá trị.

Kiểu `null` có nghĩa chúng ta biết trường đó không có giá trị, vì vậy giá trị không khả dụng, và nó biểu thị sự vắng mặt có chủ ý của một giá trị.

Array

`array` là kiểu dữ liệu có thể lưu nhiều giá trị cùng kiểu hoặc khác kiểu. Có thể định nghĩa bằng cú pháp sau:

```
const x: string[] = ['a', 'b'];
const y: Array<string> = ['a', 'b'];
const j: Array<string | number> = ['a', 1, 'b', 2]; // Union
```

TypeScript hỗ trợ mảng readonly bằng cú pháp sau:

```
const x: readonly string[] = ['a', 'b']; // Từ khóa bổ nghĩa readonly
const y: ReadonlyArray<string> = ['a', 'b'];
const j: ReadonlyArray<string | number> = ['a', 1, 'b', 2];
j.push('x'); // Không hợp lệ
```

TypeScript hỗ trợ tuple và readonly tuple:

```
const x: [string, number] = ['a', 1];
const y: readonly [string, number] = ['a', 1];
```

any

Kiểu dữ liệu `any` biểu diễn đúng nghĩa "bất kỳ" giá trị nào và là mặc định khi TypeScript không thể suy luận kiểu hoặc kiểu không được chỉ định.

Khi sử dụng `any`, trình biên dịch TypeScript bỏ qua kiểm tra kiểu, vì vậy không còn an toàn kiểu khi `any` được dùng. Nói chung, đừng sử dụng `any` để làm im trình biên dịch khi xảy ra lỗi; thay vào đó, hãy tập trung sửa lỗi, vì dùng `any` có thể phá vỡ

các hợp đồng và làm mất lợi ích của tính năng tự động hoàn thành của TypeScript.

Kiểu `any` có thể hữu ích trong quá trình chuyển đổi dần từ JavaScript sang TypeScript vì nó có thể làm im trình biên dịch.

Với dự án mới, hãy sử dụng cấu hình TypeScript `noImplicitAny`, cho phép TypeScript báo lỗi tại những nơi `any` được sử dụng hoặc suy luận.

Kiểu `any` thường là nguồn gây lỗi có thể che giấu các vấn đề thực sự với kiểu của bạn. Hãy tránh sử dụng nó nhiều nhất có thể.

Chú thích kiểu

Trên các biến được khai báo bằng `var`, `let` và `const`, có thể tùy chọn thêm một kiểu:

```
const x: number = 1;
```

TypeScript làm tốt việc suy luận kiểu, đặc biệt với các kiểu đơn giản, nên các khai báo này không cần thiết trong hầu hết trường hợp.

Trên hàm, có thể thêm chú thích kiểu cho tham số:

```
function sum(a: number, b: number) {
    return a + b;
}
```

Sau đây là ví dụ sử dụng hàm ẩn danh (còn gọi là lambda function):

```
const sum = (a: number, b: number) => a + b;
```

Có thể tránh các chú thích này khi tham số có giá trị mặc định:

```
const sum = (a = 10, b: number) => a + b;
```

Có thể thêm chú thích kiểu trả về cho hàm:

```
const sum = (a = 10, b: number): number => a + b;
```

Điều này đặc biệt hữu ích với các hàm phức tạp hơn, vì viết kiểu trả về trước phần triển khai có thể giúp bạn suy nghĩ rõ hơn về hàm.

Nói chung, hãy cân nhắc chú thích chữ ký kiểu, nhưng không cần chú thích các biến cục bộ trong thân hàm, và luôn thêm kiểu cho object literal.

Thuộc tính tùy chọn

Một đối tượng có thể chỉ định Thuộc tính tùy chọn bằng cách thêm dấu hỏi `?` vào cuối tên thuộc tính:

```
type X = {
    a: number;
    b?: number; // Tùy chọn
};
```

Có thể chỉ định giá trị mặc định khi một thuộc tính là tùy chọn:

```
type X = {
    a: number;
    b?: number;
};
const x = ({ a, b = 100 }: X) => a + b;
```

Thuộc tính readonly

Có thể ngăn việc ghi vào một thuộc tính bằng modifier `readonly`, bảo đảm thuộc tính không thể được ghi lại nhưng không cung cấp bảo đảm về tính bất biến hoàn toàn:

```
interface Y {
    readonly a: number;
}

type X = {
    readonly a: number;
};

type J = Readonly<{
    a: number;
}>;

type K = {
    readonly [index: number]: string;
};
```

Index Signature

Trong TypeScript, chúng ta có thể sử dụng `string`, `number` và `symbol` làm index signature:

```
type K = {
    [name: string | number]: string;
};
const k: K = { x: 'x', 1: 'b' };
console.log(k['x']);
console.log(k[1]);
console.log(k['1']); // Kết quả giống k[1]
```

Lưu ý rằng JavaScript tự động chuyển một index kiểu `number` thành index kiểu `string`, vì vậy `k[1]` hoặc `k["1"]` trả về cùng một giá trị.

Mở rộng kiểu

Có thể mở rộng một `interface` (sao chép các thành viên từ một kiểu khác):

```
interface X {
    a: string;
}
interface Y extends X {
    b: string;
}
```

Cũng có thể mở rộng từ nhiều kiểu:

```
interface A {
    a: string;
}
interface B {
    b: string;
}
interface Y extends A, B {
    y: string;
}
```

Từ khóa `extends` chỉ hoạt động trên interface và class; với type, hãy dùng intersection:

```
type A = {
    a: number;
};
type B = {
    b: number;
};
type C = A & B;
```

Có thể mở rộng một type bằng interface nhưng không thể làm ngược lại:

```
type A = {
    a: string;
};
interface B extends A {
    b: string;
}
```

Literal Type

Literal Type là một tập hợp một phần tử trong một kiểu tập hợp; nó định nghĩa một giá trị rất chính xác là primitive của JavaScript.

Literal Type trong TypeScript là số, chuỗi và boolean.

Ví dụ về literal:

```
const a = 'a'; // Kiểu literal chuỗi
const b = 1; // Kiểu literal số
const c = true; // Kiểu literal boolean
```

String, Numeric và Boolean Literal Type được dùng trong union, type guard và type alias. Trong ví dụ sau, bạn có thể thấy một union type alias. `O` chỉ gồm các giá trị đã chỉ định, không có chuỗi nào khác hợp lệ:

```
type O = 'a' | 'b' | 'c';
```

Suy luận literal

Literal Inference là một tính năng trong TypeScript cho phép suy luận kiểu của biến hoặc tham số dựa trên giá trị của nó.

Trong ví dụ sau, chúng ta có thể thấy TypeScript coi `x` là literal type vì giá trị không thể thay đổi về sau, trong khi `y` được suy

luận là string vì nó có thể được sửa đổi bất kỳ lúc nào sau đó.

```
const x = 'x'; // Kiểu literal 'x', vì giá trị này không thể thay đổi
let y = 'y'; // Kiểu string, vì ta có thể thay đổi giá trị này
```

Trong ví dụ sau, chúng ta có thể thấy `o.x` được suy luận là `string` (không phải literal của `a`) vì TypeScript cho rằng giá trị có thể thay đổi bất kỳ lúc nào sau đó.

```
type X = 'a' | 'b';

let o = {
    x: 'a', // Đây là chuỗi có kiểu rộng hơn
};

const fn = (x: X) => `${x}-foo`;

console.log(fn(o.x)); // Đối số có kiểu 'string' không thể gán cho tham số có kiểu 'X'
```

Như bạn có thể thấy, mã gây lỗi khi truyền `o.x` vào `fn` vì X là kiểu hẹp hơn.

Chúng ta có thể giải quyết vấn đề này bằng type assertion với `const` hoặc kiểu `X`:

```
let o = {
    x: 'a' as const,
};
```

hoặc:

```
let o = {
    x: 'a' as X,
};
```

strictNullChecks

`strictNullChecks` là một tùy chọn của trình biên dịch TypeScript thực thi việc kiểm tra null nghiêm ngặt. Khi tùy chọn này được bật, biến và tham số chỉ có thể được gán `null` hoặc `undefined` nếu chúng đã được khai báo tường minh là kiểu đó bằng union type `null | undefined`. Nếu biến hoặc tham số không được khai báo tường minh là nullable, TypeScript sẽ tạo lỗi để ngăn các lỗi thời gian chạy tiềm ẩn.

Enum

Trong TypeScript, `enum` là một tập hợp các giá trị hằng được đặt tên.

```
enum Color {
    Red = '#ff0000',
    Green = '#00ff00',
    Blue = '#0000ff',
}
```

Enum có thể được định nghĩa theo nhiều cách:

Enum số

Trong TypeScript, Numeric Enum là một Enum mà mỗi hằng được gán một giá trị số, mặc định bắt đầu từ 0.

```
enum Size {
    Small, // giá trị bắt đầu từ 0
    Medium,
    Large,
}
```

Có thể chỉ định các giá trị tùy chỉnh bằng cách gán chúng tường minh:

```
enum Size {
    Small = 10,
    Medium,
    Large,
}
console.log(Size.Medium); // 11
```

Enum chuỗi

Trong TypeScript, String enum là một Enum mà mỗi hằng được gán một giá trị chuỗi.

```
enum Language {
    English = 'EN',
    Spanish = 'ES',
}
```

Lưu ý: TypeScript cho phép sử dụng Enum không đồng nhất, trong đó thành viên chuỗi và số có thể cùng tồn tại.

Const enum

Const enum trong TypeScript là một kiểu Enum đặc biệt, trong đó tất cả giá trị đều được biết tại thời điểm biên dịch và được inline tại mọi nơi enum được sử dụng, tạo ra mã hiệu quả hơn.

```
const enum Language {
    English = 'EN',
    Spanish = 'ES',
}
console.log(Language.English);
```

Sẽ được biên dịch thành:

```
console.log('EN' /* Language.English */);
```

Ghi chú: Const Enum có các giá trị được mã hóa trực tiếp, xóa Enum khỏi đầu ra, điều này có thể hiệu quả hơn trong các thư viện khép kín nhưng nhìn chung không được mong muốn. Ngoài ra, Const enum không thể có computed member.

Ánh xạ ngược

Trong TypeScript, reverse mapping trong Enum nói đến khả năng truy xuất tên thành viên Enum từ giá trị của nó. Theo mặc định, các thành viên Enum có ánh xạ xuôi từ tên sang giá trị, nhưng có thể tạo ánh xạ ngược bằng cách đặt tường minh giá trị cho từng thành viên. Ánh xạ ngược hữu ích khi cần tra cứu một thành viên Enum theo giá trị của nó hoặc khi cần lặp qua tất cả thành viên Enum. Lưu ý rằng chỉ các thành viên enum số mới tạo ánh xạ ngược, còn các thành viên enum chuỗi hoàn toàn không được tạo ánh xạ ngược.

Enum sau:

```
enum Grade {
    A = 90,
    B = 80,
    C = 70,
    F = 'fail',
}
```

Được biên dịch thành:

```
'use strict';
var Grade;
(function (Grade) {
    Grade[(Grade['A'] = 90)] = 'A';
    Grade[(Grade['B'] = 80)] = 'B';
    Grade[(Grade['C'] = 70)] = 'C';
```

```
    Grade['F'] = 'fail';
})(Grade || (Grade = {}));
```

Do đó, ánh xạ giá trị về key hoạt động với thành viên enum số nhưng không hoạt động với thành viên enum chuỗi:

```
enum Grade {
    A = 90,
    B = 80,
    C = 70,
    F = 'fail',
}
const myGrade = Grade.A;
console.log(Grade[myGrade]); // A
console.log(Grade[90]); // A

const failGrade = Grade.F;
console.log(failGrade); // fail
console.log(Grade[failGrade]);  // Phần tử ngầm định có kiểu
'any' vì biểu thức chỉ mục không có kiểu 'number'.
```

Ambient enum

Ambient enum trong TypeScript là một kiểu Enum được định nghĩa trong tệp khai báo (*.d.ts) mà không có phần triển khai đi kèm. Nó cho phép bạn định nghĩa một tập hợp các hằng được đặt tên có thể sử dụng theo cách an toàn kiểu trên nhiều tệp khác nhau mà không cần import chi tiết triển khai trong từng tệp.

Thành viên computed và constant

Trong TypeScript, computed member là thành viên của Enum có giá trị được tính tại thời gian chạy, còn constant member là thành viên có giá trị được đặt tại thời điểm biên dịch và không thể thay đổi trong thời gian chạy. Computed member được phép

trong Enum thông thường, còn constant member được phép trong cả Enum thông thường và const enum.

```
// Các thành viên hằng
enum Color {
    Red = 1,
    Green = 5,
    Blue = Red + Green,
}
console.log(Color.Blue); // 6 được tạo tại thời điểm biên dịch

// Các thành viên được tính toán
enum Color {
    Red = 1,
    Green = Math.pow(2, 2),
    Blue = Math.floor(Math.random() * 3) + 1,
}
console.log(Color.Blue);  // số ngẫu nhiên được tạo tại thời
điểm chạy
```

Enum được biểu diễn bằng các union gồm kiểu của các thành viên. Giá trị của mỗi thành viên có thể được xác định thông qua biểu thức hằng hoặc không hằng, trong đó các thành viên có giá trị hằng được gán literal type. Để minh họa, hãy xem khai báo kiểu E và các kiểu con E.A, E.B và E.C. Trong trường hợp này, E biểu diễn union E.A | E.B | E.C.

```
const identity = (value: number) => value;

enum E {
    A = 2 * 5, // Literal số
    B = 'bar', // Literal chuỗi
     C = identity(42), // Giá trị được tính toán không tường
minh
}

console.log(E.C); //42
```

Narrowing

Narrowing trong TypeScript là quá trình tinh chỉnh kiểu của một biến bên trong một khối điều kiện. Điều này hữu ích khi làm việc với union type, nơi một biến có thể có nhiều hơn một kiểu.

TypeScript nhận biết một số cách để thu hẹp kiểu:

Type guard typeof

Type guard typeof là một type guard cụ thể trong TypeScript kiểm tra kiểu của biến dựa trên kiểu JavaScript tích hợp của nó.

```
const fn = (x: number | string) => {
    if (typeof x === 'number') {
        return x + 1; // x có kiểu number
    }
    return -1;
};
```

Thu hẹp theo truthiness

Truthiness narrowing trong TypeScript hoạt động bằng cách kiểm tra một biến là truthy hay falsy để thu hẹp kiểu tương ứng.

```
const toUpperCase = (name: string | null) => {
    if (name) {
        return name.toUpperCase();
    } else {
        return null;
    }
};
```

Thu hẹp theo phép so sánh bằng

Equality narrowing trong TypeScript hoạt động bằng cách kiểm tra một biến có bằng một giá trị cụ thể hay không để thu hẹp kiểu tương ứng.

Nó được sử dụng cùng các câu lệnh `switch` và các toán tử so sánh bằng như `===`, `!==`, `==` và `!=` để thu hẹp kiểu.

```
const checkStatus = (status: 'success' | 'error') => {
    switch (status) {
        case 'success':
            return true;
        case 'error':
            return null;
    }
};
```

Thu hẹp bằng toán tử In

Thu hẹp bằng toán tử `in` trong TypeScript là cách thu hẹp kiểu của một biến dựa trên việc một thuộc tính có tồn tại trong kiểu của biến hay không.

```
type Dog = {
    name: string;
    breed: string;
};

type Cat = {
    name: string;
    likesCream: boolean;
};

const getAnimalType = (pet: Dog | Cat) => {
    if ('breed' in pet) {
        return 'dog';
```

```
    } else {
        return 'cat';
    }
};
```

Thu hẹp bằng instanceof

Thu hẹp bằng toán tử `instanceof` trong TypeScript là cách thu hẹp kiểu của biến dựa trên hàm constructor của nó, bằng cách kiểm tra xem một đối tượng có phải là instance của một class hoặc interface nhất định hay không.

```
class Square {
    constructor(public width: number) {}
}
class Rectangle {
    constructor(
        public width: number,
        public height: number
    ) {}
}
function area(shape: Square | Rectangle) {
    if (shape instanceof Square) {
        return shape.width * shape.width;
    } else {
        return shape.width * shape.height;
    }
}
const square = new Square(5);
const rectangle = new Rectangle(5, 10);
console.log(area(square)); // 25
console.log(area(rectangle)); // 50
```

Phép gán

Thu hẹp TypeScript bằng phép gán là cách thu hẹp kiểu của một biến dựa trên giá trị được gán cho nó. Khi một biến được gán

giá trị, TypeScript suy luận kiểu của nó dựa trên giá trị đã gán và thu hẹp kiểu của biến để khớp với kiểu được suy luận.

```
let value: string | number;
value = 'hello';
if (typeof value === 'string') {
    console.log(value.toUpperCase());
}
value = 42;
if (typeof value === 'number') {
    console.log(value.toFixed(2));
}
```

Phân tích luồng điều khiển

Control Flow Analysis trong TypeScript là cách phân tích tĩnh luồng mã để suy luận kiểu của biến, cho phép trình biên dịch thu hẹp kiểu của các biến đó khi cần, dựa trên kết quả phân tích.

Trước TypeScript 4.4, phân tích luồng mã chỉ được áp dụng cho mã bên trong câu lệnh if, nhưng kể từ TypeScript 4.4, nó cũng có thể được áp dụng cho các biểu thức điều kiện và truy cập thuộc tính discriminant được tham chiếu gián tiếp thông qua biến const.

Ví dụ:

```
const f1 = (x: unknown) => {
    const isString = typeof x === 'string';
    if (isString) {
        x.length;
    }
};

const f2 = (
```

```
     obj: { kind: 'foo'; foo: string } | { kind: 'bar'; bar:
number }
) => {
    const isFoo = obj.kind === 'foo';
    if (isFoo) {
        obj.foo;
    } else {
        obj.bar;
    }
};
```

Một số ví dụ mà việc thu hẹp không xảy ra:

```
const f1 = (x: unknown) => {
    let isString = typeof x === 'string';
    if (isString) {
         x.length; // Lỗi, không thu hẹp kiểu vì isString không
phải const
    }
};

const f6 = (
     obj: { kind: 'foo'; foo: string } | { kind: 'bar'; bar:
number }
) => {
    const isFoo = obj.kind === 'foo';
    obj = obj;
    if (isFoo) {
          obj.foo; // Lỗi, không thu hẹp kiểu vì obj được gán
trong thân hàm
    }
};
```

Ghi chú: Tối đa năm cấp độ gián tiếp được phân tích trong biểu thức điều kiện.

Type Predicate

Type Predicate trong TypeScript là các hàm trả về giá trị boolean và được dùng để thu hẹp kiểu của một biến thành kiểu cụ thể hơn.

```
const isString = (value: unknown): value is string => typeof
value === 'string';

const foo = (bar: unknown) => {
    if (isString(bar)) {
        console.log(bar.toUpperCase());
    } else {
        console.log('not a string');
    }
};
```

TypeScript 5.5 tự động suy luận type predicate (như `x is T`) trong các hàm như `.filter`, nhờ đó nó biết khi các giá trị như undefined đã bị loại bỏ, tạo ra kiểu chính xác hơn và ít lỗi hơn; điều này hoạt động với các kiểm tra rõ ràng (ví dụ `x !== undefined`) nhưng không với các kiểm tra mơ hồ như `!!x`.

```
const nums = [1, null, 2].filter(x => x !== null);
```

Discriminated Union

Discriminated Union trong TypeScript là một loại union type sử dụng một thuộc tính chung, được gọi là discriminant, để thu hẹp tập hợp các kiểu có thể có của union.

```
type Square = {
    kind: 'square'; // Thuộc tính phân biệt
    size: number;
};

type Circle = {
    kind: 'circle'; // Thuộc tính phân biệt
```

```
    radius: number;
};

type Shape = Square | Circle;

const area = (shape: Shape) => {
    switch (shape.kind) {
        case 'square':
            return Math.pow(shape.size, 2);
        case 'circle':
            return Math.PI * Math.pow(shape.radius, 2);
    }
};

const square: Square = { kind: 'square', size: 5 };
const circle: Circle = { kind: 'circle', radius: 2 };

console.log(area(square)); // 25
console.log(area(circle)); // 12.566370614359172
```

Kiểu never

Khi một biến được thu hẹp thành một kiểu không thể chứa bất kỳ giá trị nào, trình biên dịch TypeScript sẽ suy luận rằng biến phải có kiểu `never`. Lý do là kiểu never biểu diễn một giá trị không bao giờ có thể được tạo ra.

```
const printValue = (val: string | number) => {
    if (typeof val === 'string') {
        console.log(val.toUpperCase());
    } else if (typeof val === 'number') {
        console.log(val.toFixed(2));
    } else {
         // val có kiểu never ở đây vì nó không thể là bất cứ thứ gì ngoài string hoặc number
        const neverVal: never = val;
        console.log(`Unexpected value: ${neverVal}`);
```

```
    }
};
```

Kiểm tra tính đầy đủ

Exhaustiveness checking là một tính năng trong TypeScript bảo đảm tất cả trường hợp có thể có của một discriminated union đều được xử lý trong câu lệnh `switch` hoặc câu lệnh `if`.

```
type Direction = 'up' | 'down';

const move = (direction: Direction) => {
    switch (direction) {
        case 'up':
            console.log('Moving up');
            break;
        case 'down':
            console.log('Moving down');
            break;
        default:
            const exhaustiveCheck: never = direction;
             console.log(exhaustiveCheck); // Dòng này sẽ không bao giờ được thực thi
    }
};
```

Kiểu `never` được dùng để bảo đảm trường hợp default là đầy đủ và TypeScript sẽ báo lỗi nếu một giá trị mới được thêm vào kiểu Direction mà không được xử lý trong câu lệnh switch.

Kiểu đối tượng

Trong TypeScript, kiểu đối tượng mô tả hình dạng của một đối tượng. Chúng chỉ định tên và kiểu của các thuộc tính của đối tượng, cũng như các thuộc tính đó là bắt buộc hay tùy chọn.

Trong TypeScript, bạn có thể định nghĩa kiểu đối tượng theo hai cách chính:

Interface định nghĩa hình dạng của một đối tượng bằng cách chỉ định tên, kiểu và tính tùy chọn của các thuộc tính.

```
interface User {
    name: string;
    age: number;
    email?: string;
}
```

Type alias, tương tự interface, định nghĩa hình dạng của một đối tượng. Tuy nhiên, nó cũng có thể tạo một kiểu tùy chỉnh mới dựa trên kiểu hiện có hoặc kết hợp các kiểu hiện có. Điều này bao gồm việc định nghĩa union type, intersection type và các kiểu phức tạp khác.

```
type Point = {
    x: number;
    y: number;
};
```

Cũng có thể định nghĩa một kiểu ẩn danh:

```
const sum = (x: { a: number; b: number }) => x.a + x.b;
console.log(sum({ a: 5, b: 1 }));
```

Tuple Type (ẩn danh)

Tuple Type là kiểu biểu diễn một mảng có số lượng phần tử cố định và các kiểu tương ứng của chúng. Tuple type bắt buộc một số lượng phần tử cụ thể và kiểu tương ứng của từng phần tử theo thứ tự cố định. Tuple type hữu ích khi bạn muốn biểu diễn

một tập hợp các giá trị có kiểu cụ thể, trong đó vị trí của mỗi phần tử trong mảng có ý nghĩa riêng.

```
type Point = [number, number];
```

Named Tuple Type (có nhãn)

Tuple type có thể bao gồm nhãn hoặc tên tùy chọn cho từng phần tử. Các nhãn này phục vụ khả năng đọc và hỗ trợ công cụ, không ảnh hưởng đến các thao tác bạn có thể thực hiện với chúng.

```
type T = string;
type Tuple1 = [T, T];
type Tuple2 = [a: T, b: T];
type Tuple3 = [a: T, T]; // Tuple có tên kết hợp với tuple ẩn danh
```

Tuple có độ dài cố định

Fixed Length Tuple là một kiểu tuple cụ thể bắt buộc số lượng phần tử cố định với các kiểu cụ thể và không cho phép thay đổi độ dài của tuple sau khi được định nghĩa.

Fixed Length Tuple hữu ích khi bạn cần biểu diễn một tập hợp giá trị với số lượng phần tử và kiểu cụ thể, đồng thời muốn bảo đảm độ dài và kiểu của tuple không bị thay đổi ngoài ý muốn.

```
const x = [10, 'hello'] as const;
x.push(2); // Lỗi
```

Union Type

Union Type là kiểu biểu diễn một giá trị có thể thuộc một trong nhiều kiểu. Union Type được biểu diễn bằng ký hiệu | giữa mỗi kiểu có thể có.

```
let x: string | number;
x = 'hello'; // Hợp lệ
x = 123; // Hợp lệ
```

Intersection Type

Intersection Type là kiểu biểu diễn một giá trị có tất cả thuộc tính của hai hoặc nhiều kiểu. Intersection Type được biểu diễn bằng ký hiệu & giữa mỗi kiểu.

```
type X = {
    a: string;
};

type Y = {
    b: string;
};

type J = X & Y; // Giao

const j: J = {
    a: 'a',
    b: 'b',
};
```

Type Indexing

Type indexing nói đến khả năng định nghĩa các kiểu có thể được index bằng một key chưa biết trước, sử dụng index signature để chỉ định kiểu cho các thuộc tính không được khai báo tường minh.

```
type Dictionary<T> = {
    [key: string]: T;
};
const myDict: Dictionary<string> = { a: 'a', b: 'b' };
console.log(myDict['a']); // Trả về a
```

Kiểu từ giá trị

Type from Value trong TypeScript nói đến việc tự động suy luận kiểu từ một giá trị hoặc biểu thức thông qua type inference.

```
const x = 'x'; // TypeScript suy luận 'x' là literal chuỗi khi dùng 'const' (bất biến), nhưng mở rộng thành 'string' khi dùng 'let' (có thể gán lại).
```

Kiểu từ giá trị trả về của hàm

Type from Func Return nói đến khả năng tự động suy luận kiểu trả về của một hàm dựa trên phần triển khai. Điều này cho phép TypeScript xác định kiểu của giá trị được hàm trả về mà không cần chú thích kiểu tường minh.

```
const add = (x: number, y: number) => x + y; // TypeScript có thể suy luận rằng kiểu trả về của hàm là number
```

Kiểu từ module

Type from Module nói đến khả năng sử dụng các giá trị được export của một module để tự động suy luận kiểu của chúng. Khi một module export một giá trị có kiểu cụ thể, TypeScript có thể dùng thông tin đó để tự động suy luận kiểu của giá trị khi nó được import vào module khác.

```
// calc.ts
export const add = (x: number, y: number) => x + y;
// index.ts
import { add } from 'calc';
const r = add(1, 2); // r có kiểu number
```

Mapped Type

Mapped Type trong TypeScript cho phép bạn tạo kiểu mới dựa trên một kiểu hiện có bằng cách biến đổi từng thuộc tính thông qua một hàm ánh xạ. Bằng cách ánh xạ các kiểu hiện có, bạn có thể tạo các kiểu mới biểu diễn cùng thông tin ở định dạng khác. Để tạo mapped type, bạn truy cập các thuộc tính của một kiểu hiện có bằng toán tử `keyof` rồi thay đổi chúng để tạo ra kiểu mới. Trong ví dụ sau:

```
type MyMappedType<T> = {
    [P in keyof T]: T[P][];
};
type MyType = {
    foo: string;
    bar: number;
};
type MyNewType = MyMappedType<MyType>;
const x: MyNewType = {
    foo: ['hello', 'world'],
    bar: [1, 2, 3],
};
```

chúng ta định nghĩa MyMappedType để ánh xạ qua các thuộc tính của T, tạo một kiểu mới trong đó mỗi thuộc tính là một mảng của kiểu ban đầu. Sử dụng nó, chúng ta tạo MyNewType để biểu diễn cùng thông tin như MyType nhưng mỗi thuộc tính là một mảng.

Modifier của Mapped Type

Mapped Type Modifier trong TypeScript cho phép biến đổi các thuộc tính trong một kiểu hiện có:

- `readonly` hoặc `+readonly`: Làm cho một thuộc tính trong mapped type chỉ đọc.

- `-readonly`: Cho phép một thuộc tính trong mapped type có thể thay đổi.
- `?`: Đánh dấu một thuộc tính trong mapped type là tùy chọn.

Ví dụ:

```
type ReadOnly<T> = { readonly [P in keyof T]: T[P] }; // Tất cả thuộc tính được đánh dấu là chỉ đọc

type Mutable<T> = { -readonly [P in keyof T]: T[P] }; // Tất cả thuộc tính được đánh dấu là có thể thay đổi

type MyPartial<T> = { [P in keyof T]?: T[P] }; // Tất cả thuộc tính được đánh dấu là tùy chọn
```

Conditional Type

Conditional Type là cách tạo một kiểu phụ thuộc vào một điều kiện, trong đó kiểu được tạo ra được xác định dựa trên kết quả của điều kiện. Chúng được định nghĩa bằng từ khóa `extends` và toán tử ba ngôi để chọn có điều kiện giữa hai kiểu.

```
type IsArray<T> = T extends any[] ? true : false;

const myArray = [1, 2, 3];
const myNumber = 42;

type IsMyArrayAnArray = IsArray<typeof myArray>; // Kiểu true
type IsMyNumberAnArray = IsArray<typeof myNumber>; // Kiểu false
```

Distributive Conditional Type

Distributive Conditional Type là một tính năng cho phép một kiểu được phân phối trên một union các kiểu bằng cách áp

dụng một phép biến đổi riêng cho từng thành viên của union. Điều này có thể đặc biệt hữu ích khi làm việc với mapped type hoặc higher-order type.

```
type Nullable<T> = T extends any ? T | null : never;
type NumberOrBool = number | boolean;
type NullableNumberOrBool = Nullable<NumberOrBool>; // number | boolean | null
```

Suy luận kiểu infer trong Conditional Type

Từ khóa `infer` được sử dụng trong conditional type để suy luận (trích xuất) kiểu của một tham số generic từ một kiểu phụ thuộc vào nó. Điều này cho phép bạn viết các định nghĩa kiểu linh hoạt và có thể tái sử dụng hơn.

```
type ElementType<T> = T extends (infer U)[] ? U : never;
type Numbers = ElementType<number[]>; // number
type Strings = ElementType<string[]>; // string
```

Conditional Type được định nghĩa sẵn

Trong TypeScript, Predefined Conditional Type là các conditional type tích hợp do ngôn ngữ cung cấp. Chúng được thiết kế để thực hiện các phép biến đổi kiểu phổ biến dựa trên đặc điểm của một kiểu nhất định.

`Exclude<UnionType, ExcludedType>`: Kiểu này loại bỏ khỏi Type tất cả các kiểu có thể gán cho ExcludedType.

`Extract<Type, Union>`: Kiểu này trích xuất từ Union tất cả các kiểu có thể gán cho Type.

`NonNullable<Type>`: Kiểu này loại bỏ null và undefined khỏi Type.

`ReturnType<Type>`: Kiểu này trích xuất kiểu trả về của một hàm Type.

`Parameters<Type>`: Kiểu này trích xuất các kiểu tham số của một hàm Type.

`Required<Type>`: Kiểu này biến tất cả thuộc tính trong Type thành bắt buộc.

`Partial<Type>`: Kiểu này biến tất cả thuộc tính trong Type thành tùy chọn.

`Readonly<Type>`: Kiểu này biến tất cả thuộc tính trong Type thành readonly.

Template Union Type

Template union type có thể được dùng để kết hợp và thao tác văn bản bên trong hệ thống kiểu, ví dụ:

```
type Status = 'active' | 'inactive';
type Products = 'p1' | 'p2';
type ProductId = `id-${Products}-${Status}`; // "id-p1-active" | "id-p1-inactive" | "id-p2-active" | "id-p2-inactive"
```

Kiểu any

Kiểu `any` là một kiểu đặc biệt (universal supertype) có thể được dùng để biểu diễn bất kỳ kiểu giá trị nào (primitive, object, array, function, error, symbol). Nó thường được sử dụng trong những tình huống kiểu của một giá trị không được biết tại thời

điểm biên dịch hoặc khi làm việc với giá trị từ API hoặc thư viện bên ngoài không có TypeScript typing.

Khi sử dụng kiểu any, bạn cho trình biên dịch TypeScript biết rằng các giá trị nên được biểu diễn mà không có bất kỳ giới hạn nào. Để tối đa hóa an toàn kiểu trong mã, hãy cân nhắc:

- Giới hạn việc sử dụng any cho các trường hợp cụ thể khi kiểu thực sự chưa biết.
- Không trả về kiểu any từ một hàm, vì điều này làm suy yếu an toàn kiểu trong mã sử dụng nó.
- Thay vì any, hãy dùng `@ts-ignore` nếu bạn cần làm im trình biên dịch.

```
let value: any;
value = true; // Hợp lệ
value = 7; // Hợp lệ
```

Kiểu unknown

Trong TypeScript, kiểu unknown biểu diễn một giá trị có kiểu chưa biết. Không giống kiểu any cho phép bất kỳ kiểu giá trị nào, unknown yêu cầu kiểm tra kiểu hoặc assertion trước khi có thể được sử dụng theo một cách cụ thể, vì vậy không có thao tác nào được phép trên unknown nếu trước tiên chưa assertion hoặc thu hẹp thành một kiểu cụ thể hơn.

Kiểu unknown chỉ có thể gán cho any và chính unknown, đồng thời là lựa chọn thay thế an toàn kiểu cho any.

```
let value: unknown;

let value1: unknown = value; // Hợp lệ
let value2: any = value; // Hợp lệ
```

```
let value3: boolean = value; // Không hợp lệ
let value4: number = value; // Không hợp lệ

const add = (a: unknown, b: unknown): number | undefined =>
     typeof a === 'number' && typeof b === 'number' ? a + b :
undefined;
console.log(add(1, 2)); // 3
console.log(add('x', 2)); // undefined
```

Kiểu void

Kiểu `void` được dùng để chỉ ra rằng một hàm không trả về giá trị.

```
const sayHello = (): void => {
    console.log('Hello!');
};
```

Kiểu dữ liệu never

Kiểu `never` biểu diễn các giá trị không bao giờ xảy ra. Nó được dùng để biểu thị các hàm hoặc biểu thức không bao giờ trả về hoặc luôn throw lỗi.

Ví dụ, một vòng lặp vô hạn:

```
const infiniteLoop = (): never => {
    while (true) {
        // làm điều gì đó
    }
};
```

Throw một lỗi:

```
const throwError = (message: string): never => {
    throw new Error(message);
```

```
};
```

Kiểu `never` hữu ích để bảo đảm an toàn kiểu và phát hiện các lỗi tiềm ẩn trong mã. Nó giúp TypeScript phân tích và suy luận các kiểu chính xác hơn khi được dùng kết hợp với các kiểu khác và câu lệnh luồng điều khiển, ví dụ:

```
type Direction = 'up' | 'down';
const move = (direction: Direction): void => {
    switch (direction) {
        case 'up':
            // di chuyển lên
            break;
        case 'down':
            // di chuyển xuống
            break;
        default:
            const exhaustiveCheck: never = direction;
                        throw  new  Error(`Unhandled  direction:
${exhaustiveCheck}`);
    }
};
```

Interface và Type

Cú pháp chung

Trong TypeScript, interface định nghĩa cấu trúc của đối tượng, chỉ định tên và kiểu của các thuộc tính hoặc phương thức mà một đối tượng phải có. Cú pháp thông thường để định nghĩa interface trong TypeScript như sau:

```
interface InterfaceName {
    property1: Type1;
    // ...
    method1(arg1: ArgType1, arg2: ArgType2): ReturnType;
```

```
    // ...
}
```

Tương tự với định nghĩa type:

```
type TypeName = {
    property1: Type1;
    // ...
    method1(arg1: ArgType1, arg2: ArgType2): ReturnType;
    // ...
};
```

`interface InterfaceName` hoặc `type TypeName`: Định nghĩa tên của interface. `property1: Type1`: Chỉ định các thuộc tính của interface cùng các kiểu tương ứng. Có thể định nghĩa nhiều thuộc tính, mỗi thuộc tính được phân tách bằng dấu chấm phẩy. `method1(arg1: ArgType1, arg2: ArgType2): ReturnType;`: Chỉ định các phương thức của interface. Phương thức được định nghĩa bằng tên, tiếp theo là danh sách tham số trong dấu ngoặc đơn và kiểu trả về. Có thể định nghĩa nhiều phương thức, mỗi phương thức được phân tách bằng dấu chấm phẩy.

Ví dụ về interface:

```
interface Person {
    name: string;
    age: number;
    greet(): void;
}
```

Ví dụ về type:

```
type TypeName = {
    property1: string;
    method1(arg1: string, arg2: string): string;
};
```

Trong TypeScript, type được dùng để định nghĩa hình dạng của dữ liệu và thực thi kiểm tra kiểu. Có một số cú pháp phổ biến để định nghĩa type trong TypeScript tùy theo trường hợp sử dụng cụ thể. Dưới đây là một số ví dụ:

Kiểu cơ bản

```
let myNumber: number = 123; // kiểu number
let myBoolean: boolean = true; // kiểu boolean
let myArray: string[] = ['a', 'b']; // mảng chuỗi
let myTuple: [string, number] = ['a', 123]; // tuple
```

Đối tượng và Interface

```
const x: { name: string; age: number } = { name: 'Simon', age:
7 };
```

Union Type và Intersection Type

```
type MyType = string | number; // Kiểu union
let myUnion: MyType = 'hello'; // Có thể là một chuỗi
myUnion = 123; // Hoặc một số

type TypeA = { name: string };
type TypeB = { age: number };
type CombinedType = TypeA & TypeB; // Kiểu giao
let myCombined: CombinedType = { name: 'John', age: 25 }; //
Đối tượng có cả thuộc tính name và age
```

Primitive Type tích hợp

TypeScript có một số primitive type tích hợp có thể được dùng để định nghĩa biến, tham số hàm và kiểu trả về:

- `number`: Biểu diễn các giá trị số, bao gồm số nguyên và số dấu phẩy động.

- `string`: Biểu diễn dữ liệu văn bản
- `boolean`: Biểu diễn các giá trị logic, có thể là true hoặc false.
- `null`: Biểu diễn sự vắng mặt của một giá trị.
- `undefined`: Biểu diễn một giá trị chưa được gán hoặc chưa được định nghĩa.
- `symbol`: Biểu diễn một định danh duy nhất. Symbol thường được dùng làm key cho thuộc tính đối tượng.
- `bigint`: Biểu diễn số nguyên với độ chính xác tùy ý.
- `any`: Biểu diễn một kiểu động hoặc chưa biết. Biến kiểu any có thể chứa giá trị của bất kỳ kiểu nào và bỏ qua kiểm tra kiểu.
- `void`: Biểu diễn sự vắng mặt của bất kỳ kiểu nào. Nó thường được dùng làm kiểu trả về của các hàm không trả về giá trị.
- `never`: Biểu diễn một kiểu cho các giá trị không bao giờ xảy ra. Nó thường được dùng làm kiểu trả về của các hàm throw lỗi hoặc đi vào vòng lặp vô hạn.

Các đối tượng JS tích hợp phổ biến

TypeScript là tập cha của JavaScript; nó bao gồm tất cả các đối tượng JavaScript tích hợp thường dùng. Bạn có thể tìm danh sách đầy đủ các đối tượng này trên trang tài liệu Mozilla Developer Network (MDN): https://developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Global_Objects

Dưới đây là danh sách một số đối tượng JavaScript tích hợp thường dùng:

- Function
- Object
- Boolean
- Error

- Number
- BigInt
- Math
- Date
- String
- RegExp
- Array
- Map
- Set
- Promise
- Intl

Overload

Function overload trong TypeScript cho phép bạn định nghĩa nhiều chữ ký hàm cho một tên hàm duy nhất, nhờ đó có thể định nghĩa các hàm được gọi theo nhiều cách. Dưới đây là một ví dụ:

```
// Các overload
function sayHi(name: string): string;
function sayHi(names: string[]): string[];

// Phần triển khai
function sayHi(name: unknown): unknown {
    if (typeof name === 'string') {
        return `Hi, ${name}!`;
    } else if (Array.isArray(name)) {
        return name.map(name => `Hi, ${name}!`);
    }
    throw new Error('Invalid value');
}

sayHi('xx'); // Hợp lệ
sayHi(['aa', 'bb']); // Hợp lệ
```

Dưới đây là một ví dụ khác về sử dụng function overload trong một `class`:

```
class Greeter {
    message: string;

    constructor(message: string) {
        this.message = message;
    }

    // overload
    sayHi(name: string): string;
    sayHi(names: string[]): ReadonlyArray<string>;

    // phần triển khai
    sayHi(name: unknown): unknown {
        if (typeof name === 'string') {
            return `${this.message}, ${name}!`;
        } else if (Array.isArray(name)) {
                    return  name.map(name  =>  `${this.message},
${name}!`);
        }
        throw new Error('value is invalid');
    }
}
console.log(new Greeter('Hello').sayHi('Simon'));
```

Hợp nhất và mở rộng

Merging và extension nói đến hai khái niệm khác nhau liên quan đến làm việc với type và interface.

Merging cho phép bạn kết hợp nhiều khai báo có cùng tên thành một định nghĩa duy nhất, ví dụ khi bạn định nghĩa một interface có cùng tên nhiều lần:

```ts
interface X {
    a: string;
}

interface X {
    b: number;
}

const person: X = {
    a: 'a',
    b: 7,
};
```

Extension nói đến khả năng mở rộng hoặc kế thừa từ type hoặc interface hiện có để tạo type hoặc interface mới. Đây là cơ chế để thêm thuộc tính hoặc phương thức vào một kiểu hiện có mà không sửa đổi định nghĩa ban đầu của nó. Ví dụ:

```ts
interface Animal {
    name: string;
    eat(): void;
}

interface Bird extends Animal {
    sing(): void;
}

const dog: Bird = {
    name: 'Bird 1',
    eat() {
        console.log('Eating');
    },
    sing() {
        console.log('Singing');
    },
};
```

Khác biệt giữa Type và Interface

Declaration merging (augmentation):

Interface hỗ trợ declaration merging, nghĩa là bạn có thể định nghĩa nhiều interface có cùng tên và TypeScript sẽ hợp nhất chúng thành một interface duy nhất với các thuộc tính và phương thức kết hợp. Ngược lại, type không hỗ trợ declaration merging. Điều này có thể hữu ích khi bạn muốn thêm chức năng hoặc tùy chỉnh các kiểu hiện có mà không sửa đổi định nghĩa ban đầu, hoặc vá các kiểu bị thiếu hay không chính xác.

```
interface A {
    x: string;
}
interface A {
    y: string;
}
const j: A = {
    x: 'xx',
    y: 'yy',
};
```

Mở rộng type/interface khác:

Cả type và interface đều có thể mở rộng type/interface khác, nhưng cú pháp khác nhau. Với interface, bạn sử dụng từ khóa `extends` để kế thừa thuộc tính và phương thức từ interface khác. Tuy nhiên, interface không thể mở rộng một kiểu phức tạp như union type.

```
interface A {
    x: string;
    y: number;
}
```

```
interface B extends A {
    z: string;
}
const car: B = {
    x: 'x',
    y: 123,
    z: 'z',
};
```

Với type, bạn sử dụng toán tử & để kết hợp nhiều kiểu thành một kiểu duy nhất (intersection).

```
interface A {
    x: string;
    y: number;
}

type B = A & {
    j: string;
};

const c: B = {
    x: 'x',
    y: 123,
    j: 'j',
};
```

Union Type và Intersection Type:

Type linh hoạt hơn khi định nghĩa Union Type và Intersection Type. Với từ khóa `type`, bạn có thể dễ dàng tạo union type bằng toán tử `|` và intersection type bằng toán tử `&`. Trong khi interface cũng có thể biểu diễn union type một cách gián tiếp, chúng không có hỗ trợ tích hợp cho intersection type.

```
type Department = 'dep-x' | 'dep-y'; // Union
```

```
type Person = {
    name: string;
    age: number;
};

type Employee = {
    id: number;
    department: Department;
};

type EmployeeInfo = Person & Employee; // Giao
```

Ví dụ với interface:

```
interface A {
    x: 'x';
}
interface B {
    y: 'y';
}

type C = A | B; // Union của các interface
```

Class

Cú pháp class thông dụng

Từ khóa `class` được sử dụng trong TypeScript để định nghĩa một class. Dưới đây là một ví dụ:

```
class Person {
    private name: string;
    private age: number;
    constructor(name: string, age: number) {
        this.name = name;
        this.age = age;
    }
```

```
    public sayHi(): void {
        console.log(
                    `Hello, my name is ${this.name} and I am
${this.age} years old.`
        );
    }
}
```

Từ khóa `class` được dùng để định nghĩa một class có tên "Person".

Class có hai thuộc tính private: name kiểu `string` và age kiểu `number`.

Constructor được định nghĩa bằng từ khóa `constructor`. Nó nhận name và age làm tham số và gán chúng cho các thuộc tính tương ứng.

Class có một phương thức `public` tên là sayHi, phương thức này ghi ra một lời chào.

Để tạo một instance của class trong TypeScript, bạn có thể dùng từ khóa `new`, theo sau là tên class rồi dấu ngoặc đơn `()`. Ví dụ:

```
const myObject = new Person('John Doe', 25);
myObject.sayHi(); // Kết quả: Hello, my name is John Doe and I
am 25 years old.
```

Constructor

Constructor là các phương thức đặc biệt trong một class, được dùng để khởi tạo các thuộc tính của đối tượng khi một instance của class được tạo.

```
class Person {
    public name: string;
```

```
    public age: number;

    constructor(name: string, age: number) {
        this.name = name;
        this.age = age;
    }

    sayHello() {
        console.log(
            `Hello, my name is ${this.name} and I'm ${this.age}
years old.`
        );
    }
}

const john = new Person('Simon', 17);
john.sayHello();
```

Có thể overload một constructor bằng cú pháp sau:

```
type Sex = 'm' | 'f';

class Person {
    name: string;
    age: number;
    sex: Sex;

    constructor(name: string, age: number, sex?: Sex);
    constructor(name: string, age: number, sex: Sex) {
        this.name = name;
        this.age = age;
        this.sex = sex ?? 'm';
    }
}

const p1 = new Person('Simon', 17);
const p2 = new Person('Alice', 22, 'f');
```

Trong TypeScript, có thể định nghĩa nhiều overload cho constructor, nhưng chỉ có thể có một phần triển khai và phần này phải tương thích với tất cả overload; có thể đạt được điều này bằng cách sử dụng tham số tùy chọn.

```
class Person {
    name: string;
    age: number;

    constructor();
    constructor(name: string);
    constructor(name: string, age: number);
    constructor(name?: string, age?: number) {
        this.name = name ?? 'Unknown';
        this.age = age ?? 0;
    }

    displayInfo() {
        console.log(`Name: ${this.name}, Age: ${this.age}`);
    }
}

const person1 = new Person();
person1.displayInfo(); // Name: Unknown, Age: 0

const person2 = new Person('John');
person2.displayInfo(); // Name: John, Age: 0

const person3 = new Person('Jane', 25);
person3.displayInfo(); // Name: Jane, Age: 25
```

Constructor private và protected

Trong TypeScript, constructor có thể được đánh dấu là private hoặc protected, điều này hạn chế khả năng truy cập và sử dụng của chúng.

Private Constructor: Chỉ có thể được gọi bên trong chính class đó. Private constructor thường được dùng trong các tình huống bạn muốn thực thi singleton pattern hoặc giới hạn việc tạo instance vào một factory method bên trong class.

Protected Constructor: Protected constructor hữu ích khi bạn muốn tạo một base class không nên được khởi tạo trực tiếp nhưng có thể được mở rộng bởi các subclass.

```
class BaseClass {
    protected constructor() {}
}

class DerivedClass extends BaseClass {
    private value: number;

    constructor(value: number) {
        super();
        this.value = value;
    }
}

// Việc cố tạo trực tiếp một thể hiện của lớp cơ sở sẽ gây ra lỗi
// const baseObj = new BaseClass(); // Lỗi: Hàm khởi tạo của lớp 'BaseClass' là protected.

// Tạo một thể hiện của lớp dẫn xuất
const derivedObj = new DerivedClass(10);
```

Access Modifier

Access Modifier `private`, `protected` và `public` được dùng để kiểm soát tính hiển thị và khả năng truy cập của các thành viên class, như thuộc tính và phương thức, trong các class TypeScript. Các modifier này rất quan trọng để thực thi tính đóng gói và thiết

lập ranh giới cho việc truy cập và sửa đổi trạng thái nội bộ của class.

Modifier `private` giới hạn quyền truy cập thành viên class chỉ trong class chứa nó.

Modifier `protected` cho phép truy cập thành viên class trong class chứa nó và các class dẫn xuất.

Modifier `public` cung cấp quyền truy cập không hạn chế vào thành viên class, cho phép truy cập từ bất kỳ đâu.

Get và Set

Getter và setter là các phương thức đặc biệt cho phép bạn định nghĩa hành vi truy cập và sửa đổi tùy chỉnh cho thuộc tính class. Chúng cho phép đóng gói trạng thái nội bộ của đối tượng và cung cấp logic bổ sung khi lấy hoặc đặt giá trị của thuộc tính. Trong TypeScript, getter và setter được định nghĩa lần lượt bằng từ khóa `get` và `set`. Dưới đây là một ví dụ:

```
class MyClass {
    private _myProperty: string;

    constructor(value: string) {
        this._myProperty = value;
    }
    get myProperty(): string {
        return this._myProperty;
    }
    set myProperty(value: string) {
        this._myProperty = value;
    }
}
```

Auto-Accessor trong Class

TypeScript phiên bản 4.9 bổ sung hỗ trợ auto-accessor, một tính năng ECMAScript sắp tới. Chúng giống thuộc tính class nhưng được khai báo bằng từ khóa "accessor".

```
class Animal {
    accessor name: string;

    constructor(name: string) {
        this.name = name;
    }
}
```

Auto-accessor được "de-sugared" thành các accessor `get` và `set` private, hoạt động trên một thuộc tính không thể truy cập.

```
class Animal {
    #__name: string;

    get name() {
        return this.#__name;
    }
    set name(value: string) {
        this.#__name = value;
    }

    constructor(name: string) {
        this.name = name;
    }
}
```

this

Trong TypeScript, từ khóa `this` tham chiếu đến instance hiện tại của một class bên trong các phương thức hoặc constructor của

nó. Nó cho phép bạn truy cập và sửa đổi các thuộc tính và phương thức của class từ trong phạm vi của chính class đó. Nó cung cấp cách truy cập và thao tác trạng thái nội bộ của một đối tượng bên trong các phương thức của chính đối tượng đó.

```
class Person {
    private name: string;
    constructor(name: string) {
        this.name = name;
    }
    public introduce(): void {
        console.log(`Hello, my name is ${this.name}.`);
    }
}

const person1 = new Person('Alice');
person1.introduce(); // Hello, my name is Alice.
```

Parameter Property

Parameter property cho phép bạn khai báo và khởi tạo thuộc tính class trực tiếp trong các tham số constructor, tránh mã boilerplate. Ví dụ:

```
class Person {
    constructor(
        private name: string,
        public age: number
    ) {
        // Các từ khóa "private" và "public" trong hàm khởi tạo
          // tự động khai báo và khởi tạo các thuộc tính tương
ứng của lớp.
    }
    public introduce(): void {
        console.log(
                    `Hello, my name is ${this.name} and I am
${this.age} years old.`
```

```
        );
    }
}
const person = new Person('Alice', 25);
person.introduce();
```

Abstract Class

Abstract Class được sử dụng trong TypeScript chủ yếu cho kế thừa. Nó cung cấp cách định nghĩa các thuộc tính và phương thức chung có thể được các subclass kế thừa. Điều này hữu ích khi bạn muốn định nghĩa hành vi chung và bắt buộc các subclass triển khai một số phương thức nhất định. Chúng cung cấp cách tạo một hệ phân cấp class, trong đó abstract base class cung cấp interface dùng chung và chức năng chung cho các subclass.

```
abstract class Animal {
    protected name: string;

    constructor(name: string) {
        this.name = name;
    }

    abstract makeSound(): void;
}

class Cat extends Animal {
    makeSound(): void {
        console.log(`${this.name} meows.`);
    }
}

const cat = new Cat('Whiskers');
cat.makeSound(); // Kết quả: Whiskers meows.
```

Với Generic

Class với generic cho phép bạn định nghĩa các class có thể tái sử dụng và làm việc với nhiều kiểu khác nhau.

```
class Container<T> {
    private item: T;

    constructor(item: T) {
        this.item = item;
    }

    getItem(): T {
        return this.item;
    }

    setItem(item: T): void {
        this.item = item;
    }
}

const container1 = new Container<number>(42);
console.log(container1.getItem()); //  42

const container2 = new Container<string>('Hello');
container2.setItem('World');
console.log(container2.getItem()); // World
```

Decorator

Decorator cung cấp cơ chế để thêm metadata, sửa đổi hành vi, xác thực hoặc mở rộng chức năng của phần tử đích. Chúng là các hàm được thực thi tại thời gian chạy. Có thể áp dụng nhiều decorator cho một khai báo.

Decorator là tính năng thử nghiệm và các ví dụ sau chỉ tương thích với TypeScript phiên bản 5 trở lên khi dùng ES6.

Với các phiên bản TypeScript trước 5, cần bật chúng bằng thuộc tính `experimentalDecorators` trong `tsconfig.json` hoặc bằng `--experimentalDecorators` trên dòng lệnh (nhưng ví dụ sau sẽ không hoạt động).

Một số trường hợp sử dụng decorator phổ biến gồm:

- Theo dõi thay đổi thuộc tính.
- Theo dõi lời gọi phương thức.
- Thêm thuộc tính hoặc phương thức bổ sung.
- Xác thực tại thời gian chạy.
- Tự động serialization và deserialization.
- Logging.
- Authorization và authentication.
- Bảo vệ khỏi lỗi.

Lưu ý: Decorator của phiên bản 5 không cho phép trang trí tham số.

Các loại decorator:

Class Decorator

Class Decorator hữu ích để mở rộng một class hiện có, chẳng hạn thêm thuộc tính hoặc phương thức, hoặc thu thập các instance của một class. Trong ví dụ sau, chúng ta thêm phương thức `toString` chuyển class thành biểu diễn chuỗi.

```
type Constructor<T = {}> = new (...args: any[]) => T;

function toString<Class extends Constructor>(
```

```
    Value: Class,
    context: ClassDecoratorContext<Class>
) {
    return class extends Value {
        constructor(...args: any[]) {
            super(...args);
            console.log(JSON.stringify(this));
            console.log(JSON.stringify(context));
        }
    };
}

@toString
class Person {
    name: string;

    constructor(name: string) {
        this.name = name;
    }

    greet() {
        return 'Hello, ' + this.name;
    }
}
const person = new Person('Simon');
/* In ra:
{"name":"Simon"}
{"kind":"class","name":"Person"}
*/
```

Property Decorator

Property decorator hữu ích để sửa đổi hành vi của một thuộc tính, chẳng hạn thay đổi giá trị khởi tạo. Trong đoạn mã sau, chúng ta có một script đặt một thuộc tính luôn ở dạng chữ hoa:

```
function upperCase<T>(
    target: undefined,
    context: ClassFieldDecoratorContext<T, string>
) {
    return function (this: T, value: string) {
        return value.toUpperCase();
    };
}

class MyClass {
    @upperCase
    prop1 = 'hello!';
}

console.log(new MyClass().prop1); // In ra: HELLO!
```

Method Decorator

Method decorator cho phép bạn thay đổi hoặc tăng cường hành vi của phương thức. Dưới đây là ví dụ về một logger đơn giản:

```
function log<This, Args extends any[], Return>(
    target: (this: This, ...args: Args) => Return,
    context: ClassMethodDecoratorContext<
        This,
        (this: This, ...args: Args) => Return
    >
) {
    const methodName = String(context.name);

       function replacementMethod(this: This, ...args: Args):
Return {
        console.log(`LOG: Entering method '${methodName}'.`);
        const result = target.call(this, ...args);
        console.log(`LOG: Exiting method '${methodName}'.`);
        return result;
    }
```

```
    return replacementMethod;
}

class MyClass {
    @log
    sayHello() {
        console.log('Hello!');
    }
}

new MyClass().sayHello();
```

Nó ghi ra:

```
LOG: Entering method 'sayHello'.
Hello!
LOG: Exiting method 'sayHello'.
```

Getter và Setter Decorator

Getter và setter decorator cho phép bạn thay đổi hoặc tăng cường hành vi của accessor trong class. Chúng hữu ích, chẳng hạn để xác thực việc gán thuộc tính. Dưới đây là một ví dụ đơn giản về getter decorator:

```
function range<This, Return extends number>(min: number, max:
number) {
    return function (
        target: (this: This) => Return,
        context: ClassGetterDecoratorContext<This, Return>
    ) {
        return function (this: This): Return {
            const value = target.call(this);
            if (value < min || value > max) {
                throw 'Invalid';
            }
            Object.defineProperty(this, context.name, {
```

```
                value,
                enumerable: true,
            });
            return value;
        };
    };
}

class MyClass {
    private _value = 0;

    constructor(value: number) {
        this._value = value;
    }
    @range(1, 100)
    get getValue(): number {
        return this._value;
    }
}

const obj = new MyClass(10);
console.log(obj.getValue); // Hợp lệ: 10

const obj2 = new MyClass(999);
console.log(obj2.getValue); // Ném lỗi: Invalid!
```

Metadata của Decorator

Decorator Metadata đơn giản hóa quá trình để decorator áp dụng và sử dụng metadata trong bất kỳ class nào. Chúng có thể truy cập một thuộc tính metadata mới trên đối tượng context, thuộc tính này có thể đóng vai trò như key cho cả primitive và object. Có thể truy cập thông tin metadata trên class thông qua `Symbol.metadata`.

Metadata có thể được dùng cho nhiều mục đích như debugging, serialization hoặc dependency injection với decorator.

```
//@ts-ignore
Symbol.metadata ??= Symbol('Symbol.metadata'); // Polyfill đơn giản

type Context =
    | ClassFieldDecoratorContext
    | ClassAccessorDecoratorContext
     | ClassMethodDecoratorContext; // Ngữ cảnh chứa metadata của thuộc tính: DecoratorMetadata

function setMetadata(_target: any, context: Context) {
    // Gán một giá trị nguyên thủy cho đối tượng metadata
    context.metadata[context.name] = true;
}

class MyClass {
    @setMetadata
    a = 123;

    @setMetadata
    accessor b = 'b';

    @setMetadata
    fn() {}
}

const metadata = MyClass[Symbol.metadata]; // Lấy thông tin metadata

console.log(JSON.stringify(metadata));                    // {"bar":true,"baz":true,"foo":true}
```

Kế thừa

Kế thừa nói đến cơ chế mà một class có thể kế thừa thuộc tính và phương thức từ một class khác, được gọi là base class hoặc superclass. Class dẫn xuất, còn gọi là child class hoặc subclass, có thể mở rộng và chuyên biệt hóa chức năng của base class bằng cách thêm thuộc tính và phương thức mới hoặc override các thành phần hiện có.

```
class Animal {
    name: string;

    constructor(name: string) {
        this.name = name;
    }

    speak(): void {
        console.log('The animal makes a sound');
    }
}

class Dog extends Animal {
    breed: string;

    constructor(name: string, breed: string) {
        super(name);
        this.breed = breed;
    }

    speak(): void {
        console.log('Woof! Woof!');
    }
}

// Tạo một thể hiện của lớp cơ sở
const animal = new Animal('Generic Animal');
animal.speak(); // The animal makes a sound

// Tạo một thể hiện của lớp dẫn xuất
```

```
const dog = new Dog('Max', 'Labrador');
dog.speak(); // Woof! Woof!"
```

TypeScript không hỗ trợ đa kế thừa theo nghĩa truyền thống và thay vào đó chỉ cho phép kế thừa từ một base class duy nhất. TypeScript hỗ trợ nhiều interface. Một interface có thể định nghĩa hợp đồng cho cấu trúc của một đối tượng và một class có thể implement nhiều interface. Điều này cho phép class kế thừa hành vi và cấu trúc từ nhiều nguồn.

```
interface Flyable {
    fly(): void;
}

interface Swimmable {
    swim(): void;
}

class FlyingFish implements Flyable, Swimmable {
    fly() {
        console.log('Flying...');
    }

    swim() {
        console.log('Swimming...');
    }
}

const flyingFish = new FlyingFish();
flyingFish.fly();
flyingFish.swim();
```

Từ khóa `class` trong TypeScript, tương tự JavaScript, thường được gọi là syntactic sugar. Nó được giới thiệu trong ECMAScript 2015 (ES6) để cung cấp cú pháp quen thuộc hơn cho việc tạo và làm việc với đối tượng theo cách dựa trên class.

Tuy nhiên, điều quan trọng cần lưu ý là TypeScript, với tư cách tập cha của JavaScript, cuối cùng được biên dịch xuống JavaScript, vốn về cốt lõi vẫn dựa trên prototype.

Thành viên static

TypeScript có các thành viên static. Để truy cập thành viên static của một class, bạn có thể dùng tên class theo sau bởi dấu chấm mà không cần tạo đối tượng.

```
class OfficeWorker {
    static memberCount: number = 0;

    constructor(private name: string) {
        OfficeWorker.memberCount++;
    }
}

const w1 = new OfficeWorker('James');
const w2 = new OfficeWorker('Simon');
const total = OfficeWorker.memberCount;
console.log(total); // 2
```

Khởi tạo thuộc tính

Có một số cách để khởi tạo thuộc tính cho class trong TypeScript:

Inline:

Trong ví dụ sau, các giá trị ban đầu này sẽ được sử dụng khi một instance của class được tạo.

```
class MyClass {
    property1: string = 'default value';
```

```
    property2: number = 42;
}
```

Trong constructor:

```
class MyClass {
    property1: string;
    property2: number;

    constructor() {
        this.property1 = 'default value';
        this.property2 = 42;
    }
}
```

Sử dụng tham số constructor:

```
class MyClass {
    constructor(
        private property1: string = 'default value',
        public property2: number = 42
    ) {
        // Không cần gán tường minh các giá trị cho thuộc tính.
    }
    log() {
        console.log(this.property2);
    }
}
const x = new MyClass();
x.log();
```

Method overloading

Method overloading cho phép một class có nhiều phương thức cùng tên nhưng với kiểu tham số khác nhau hoặc số lượng tham số khác nhau. Điều này cho phép chúng ta gọi một

phương thức theo nhiều cách dựa trên các đối số được truyền vào.

```
class MyClass {
    add(a: number, b: number): number; // Chữ ký overload 1
    add(a: string, b: string): string; // Chữ ký overload 2

      add(a: number | string, b: number | string): number |
string {
        if (typeof a === 'number' && typeof b === 'number') {
            return a + b;
        }
        if (typeof a === 'string' && typeof b === 'string') {
            return a.concat(b);
        }
        throw new Error('Invalid arguments');
    }
}

const r = new MyClass();
console.log(r.add(10, 5)); // In ra 15
```

Generic

Generic cho phép bạn tạo các component và hàm có thể tái sử dụng, hoạt động với nhiều kiểu. Với generic, bạn có thể tham số hóa kiểu, hàm và interface, cho phép chúng hoạt động trên các kiểu khác nhau mà không cần chỉ định tường minh từ trước.

Generic cho phép bạn làm cho mã linh hoạt và có thể tái sử dụng hơn.

Generic Type

Để định nghĩa generic type, bạn sử dụng dấu ngoặc nhọn (<>) để chỉ định các tham số kiểu, ví dụ:

```
function identity<T>(arg: T): T {
    return arg;
}
const a = identity('x');
const b = identity(123);

const getLen = <T,>(data: ReadonlyArray<T>) => data.length;
const len = getLen([1, 2, 3]);
```

Generic Class

Generic cũng có thể được áp dụng cho class; theo cách này, class có thể làm việc với nhiều kiểu bằng cách sử dụng tham số kiểu. Điều này hữu ích để tạo các định nghĩa class có thể tái sử dụng, hoạt động trên các kiểu dữ liệu khác nhau trong khi vẫn duy trì an toàn kiểu.

```
class Container<T> {
    private item: T;

    constructor(item: T) {
        this.item = item;
    }

    getItem(): T {
        return this.item;
    }
}

const numberContainer = new Container<number>(123);
console.log(numberContainer.getItem()); // 123

const stringContainer = new Container<string>('hello');
console.log(stringContainer.getItem()); // hello
```

Ràng buộc Generic

Các tham số generic có thể được giới hạn bằng từ khóa `extends` theo sau bởi một type hoặc interface mà tham số kiểu phải thỏa mãn.

Trong ví dụ sau, `T` phải có thuộc tính `length` được định kiểu đúng để hợp lệ:

```
const printLen = <T extends { length: number }>(value: T): void
=> {
    console.log(value.length);
};

printLen('Hello'); // 5
printLen([1, 2, 3]); // 3
printLen({ length: 10 }); // 10
printLen(123); // Không hợp lệ
```

Một tính năng generic đáng chú ý được giới thiệu trong phiên bản 3.4 RC là suy luận kiểu cho higher-order function, giúp lan truyền các generic type argument:

```
declare function pipe<A extends any[], B, C>(
    ab: (...args: A) => B,
    bc: (b: B) => C
): (...args: A) => C;

declare function list<T>(a: T): T[];
declare function box<V>(x: V): { value: V };

const listBox = pipe(list, box); // <T>(a: T) => { value: T[] }
const boxList = pipe(box, list); // <V>(x: V) => { value: V }[]
```

Chức năng này giúp việc lập trình theo phong cách pointfree an toàn kiểu trở nên dễ dàng hơn, một phong cách phổ biến trong lập trình hàm.

Thu hẹp theo ngữ cảnh cho Generic

Contextual narrowing cho generic là cơ chế trong TypeScript cho phép trình biên dịch thu hẹp kiểu của một tham số generic dựa trên ngữ cảnh mà nó được sử dụng. Điều này hữu ích khi làm việc với generic type trong các câu lệnh điều kiện:

```
function process<T>(value: T): void {
    if (typeof value === 'string') {
        // Giá trị được thu hẹp thành kiểu 'string'
        console.log(value.length);
    } else if (typeof value === 'number') {
        // Giá trị được thu hẹp thành kiểu 'number'
        console.log(value.toFixed(2));
    }
}

process('hello'); // 5
process(3.14159); // 3.14
```

Kiểu cấu trúc bị xóa

Trong TypeScript, đối tượng không cần phải khớp với một kiểu cụ thể, chính xác. Ví dụ, nếu chúng ta tạo một đối tượng đáp ứng các yêu cầu của một interface, chúng ta có thể sử dụng đối tượng đó ở những nơi cần interface ấy, ngay cả khi giữa chúng không có liên kết tường minh. Ví dụ:

```
type NameProp1 = {
    prop1: string;
};

function log(x: NameProp1) {
    console.log(x.prop1);
}
```

```
const obj = {
    prop2: 123,
    prop1: 'Origin',
};

log(obj); // Hợp lệ
```

Namespace

Trong TypeScript, namespace được dùng để tổ chức mã thành các container logic, ngăn xung đột tên và cung cấp cách nhóm các đoạn mã liên quan với nhau. Việc sử dụng từ khóa `export` cho phép truy cập namespace từ bên ngoài module.

```
export namespace MyNamespace {
    export interface MyInterface1 {
        prop1: boolean;
    }
    export interface MyInterface2 {
        prop2: string;
    }
}

const a: MyNamespace.MyInterface1 = {
    prop1: true,
};
```

Symbol

Symbol là một kiểu dữ liệu nguyên thủy biểu diễn một giá trị bất biến được bảo đảm là duy nhất trên toàn cục trong suốt vòng đời của chương trình.

Symbol có thể được dùng làm key cho thuộc tính đối tượng và cung cấp cách tạo các thuộc tính không enumerable.

```ts
const key1: symbol = Symbol('key1');
const key2: symbol = Symbol('key2');

const obj = {
    [key1]: 'value 1',
    [key2]: 'value 2',
};

console.log(obj[key1]); // value 1
console.log(obj[key2]); // value 2
```

Trong WeakMap và WeakSet, symbol hiện được phép dùng làm key.

Triple-Slash Directive

Triple-slash directive là các comment đặc biệt cung cấp chỉ dẫn cho trình biên dịch về cách xử lý một tệp. Các directive này bắt đầu bằng ba dấu gạch chéo liên tiếp (`///`), thường được đặt ở đầu tệp TypeScript và không ảnh hưởng đến hành vi thời gian chạy.

Triple-slash directive được dùng để tham chiếu dependency bên ngoài, chỉ định hành vi tải module, bật hoặc tắt một số tính năng của trình biên dịch và nhiều mục đích khác. Một vài ví dụ:

Tham chiếu một tệp khai báo:

```ts
/// <reference path="path/to/declaration/file.d.ts" />
```

Chỉ định định dạng module:

```ts
/// <amd|commonjs|system|umd|es6|es2015|none>
```

Bật các tùy chọn trình biên dịch, trong ví dụ sau là strict mode:

```
/// <strict|noImplicitAny|noUnusedLocals|noUnusedParameters>
```

Thao tác kiểu

Tạo kiểu từ kiểu

Có thể tạo các kiểu mới bằng cách kết hợp, thao tác hoặc biến đổi các kiểu hiện có.

Intersection Type (`&`):

Cho phép bạn kết hợp nhiều kiểu thành một kiểu duy nhất:

```
type A = { foo: number };
type B = { bar: string };
type C = A & B; // Giao của A và B
const obj: C = { foo: 42, bar: 'hello' };
```

Union Type (`|`):

Cho phép bạn định nghĩa một kiểu có thể là một trong nhiều kiểu:

```
type Result = string | number;
const value1: Result = 'hello';
const value2: Result = 42;
```

Mapped Type:

Cho phép bạn biến đổi các thuộc tính của một kiểu hiện có để tạo kiểu mới:

```
type Mutable<T> = {
    readonly [P in keyof T]: T[P];
};
type Person = {
```

```
    name: string;
    age: number;
};
type ImmutablePerson = Mutable<Person>; // Các thuộc tính trở
thành chỉ đọc
```

Conditional type:

Cho phép bạn tạo kiểu dựa trên một số điều kiện:

```
type ExtractParam<T> = T extends (param: infer P) => any ? P :
never;
type MyFunction = (name: string) => number;
type ParamType = ExtractParam<MyFunction>; // string
```

Indexed Access Type

Trong TypeScript, có thể truy cập và thao tác kiểu của các thuộc tính bên trong một kiểu khác bằng index, `Type[Key]`.

```
type Person = {
    name: string;
    age: number;
};

type AgeType = Person['age']; // number

type MyTuple = [string, number, boolean];
type MyType = MyTuple[2]; // boolean
```

Utility Type

Có thể dùng một số utility type tích hợp để thao tác kiểu; dưới đây là danh sách các loại được sử dụng phổ biến nhất:

Awaited<T>

Tạo một kiểu gỡ bọc đệ quy các kiểu Promise.

```
type A = Awaited<Promise<string>>; // string
```

Partial<T>

Tạo một kiểu với tất cả thuộc tính của T được đặt thành tùy chọn.

```
type Person = {
    name: string;
    age: number;
};

type A = Partial<Person>; // { name?: string | undefined; age?:
number | undefined; }
```

Required<T>

Tạo một kiểu với tất cả thuộc tính của T được đặt thành bắt buộc.

```
type Person = {
    name?: string;
    age?: number;
};

type A = Required<Person>; // { name: string; age: number; }
```

Readonly<T>

Tạo một kiểu với tất cả thuộc tính của T được đặt thành readonly.

```
type Person = {
    name: string;
```

```
    age: number;
};

type A = Readonly<Person>;

const a: A = { name: 'Simon', age: 17 };
a.name = 'John'; // Không hợp lệ
```

Record<K, T>

Tạo một kiểu với một tập hợp thuộc tính K có kiểu T.

```
type Product = {
    name: string;
    price: number;
};

const products: Record<string, Product> = {
    apple: { name: 'Apple', price: 0.5 },
    banana: { name: 'Banana', price: 0.25 },
};

console.log(products.apple); // { name: 'Apple', price: 0.5 }
```

Pick<T, K>

Tạo một kiểu bằng cách lấy các thuộc tính K đã chỉ định từ T.

```
type Product = {
    name: string;
    price: number;
};

type Price = Pick<Product, 'price'>; // { price: number; }
```

Omit<T, K>

Tạo một kiểu bằng cách loại bỏ các thuộc tính K đã chỉ định khỏi T.

```
type Product = {
    name: string;
    price: number;
};

type Name = Omit<Product, 'price'>; // { name: string; }
```

Exclude<T, U>

Tạo một kiểu bằng cách loại trừ tất cả giá trị kiểu U khỏi T.

```
type Union = 'a' | 'b' | 'c';
type MyType = Exclude<Union, 'a' | 'c'>; // b
```

Extract<T, U>

Tạo một kiểu bằng cách trích xuất tất cả giá trị kiểu U từ T.

```
type Union = 'a' | 'b' | 'c';
type MyType = Extract<Union, 'a' | 'c'>; // a | c
```

NonNullable<T>

Tạo một kiểu bằng cách loại trừ null và undefined khỏi T.

```
type Union = 'a' | null | undefined | 'b';
type MyType = NonNullable<Union>; // 'a' | 'b'
```

Parameters<T>

Trích xuất các kiểu tham số của function type T.

```
type Func = (a: string, b: number) => void;
type MyType = Parameters<Func>; // [a: string, b: number]
```

ConstructorParameters<T>

Trích xuất các kiểu tham số của constructor function type T.

```
class Person {
    constructor(
        public name: string,
        public age: number
    ) {}
}
type PersonConstructorParams = ConstructorParameters<typeof
Person>; // [name: string, age: number]
const params: PersonConstructorParams = ['John', 30];
const person = new Person(...params);
console.log(person); // Person { name: 'John', age: 30 }
```

ReturnType<T>

Trích xuất kiểu trả về của function type T.

```
type Func = (name: string) => number;
type MyType = ReturnType<Func>; // number
```

InstanceType<T>

Trích xuất instance type của class type T.

```
class Person {
    name: string;

    constructor(name: string) {
        this.name = name;
    }
```

```
    sayHello() {
        console.log(`Hello, my name is ${this.name}!`);
    }
}

type PersonInstance = InstanceType<typeof Person>;

const person: PersonInstance = new Person('John');

person.sayHello(); // Hello, my name is John!
```

ThisParameterType<T>

Trích xuất kiểu của tham số 'this' từ function type T.

```
interface Person {
    name: string;
    greet(this: Person): void;
}
type PersonThisType = ThisParameterType<Person['greet']>; // Person
```

OmitThisParameter<T>

Loại bỏ tham số 'this' khỏi function type T.

```
function capitalize(this: String) {
                        return      this[0].toUpperCase()      +
this.substring(1).toLowerCase();
}

type CapitalizeType = OmitThisParameter<typeof capitalize>; // () => string
```

ThisType<T>

Đóng vai trò marker cho kiểu `this` theo ngữ cảnh.

```
type Logger = {
    log: (error: string) => void;
};

let helperFunctions: { [name: string]: Function } &
ThisType<Logger> = {
    hello: function () {
        this.log('some error'); // Hợp lệ vì "log" là một phần
của "this".
        this.update(); // Không hợp lệ
    },
};
```

Uppercase<T>

Chuyển tên của kiểu đầu vào T thành chữ hoa.

```
type MyType = Uppercase<'abc'>; // "ABC"
```

Lowercase<T>

Chuyển tên của kiểu đầu vào T thành chữ thường.

```
type MyType = Lowercase<'ABC'>; // "abc"
```

Capitalize<T>

Viết hoa chữ cái đầu trong tên của kiểu đầu vào T.

```
type MyType = Capitalize<'abc'>; // "Abc"
```

Uncapitalize<T>

Chuyển chữ cái đầu trong tên của kiểu đầu vào T thành chữ thường.

```
type MyType = Uncapitalize<'Abc'>; // "abc"
```

NoInfer<T>

NoInfer là một utility type được thiết kế để chặn việc tự động suy luận kiểu trong phạm vi của một hàm generic.

Ví dụ:

```
// Suy luận kiểu tự động trong phạm vi của một hàm generic.
function fn<T extends string>(x: T[], y: T) {
    return x.concat(y);
}
const r = fn(['a', 'b'], 'c'); // Kiểu ở đây là ("a" | "b" |
"c")[]
```

Với NoInfer:

```
// Hàm ví dụ dùng NoInfer để ngăn suy luận kiểu
function fn2<T extends string>(x: T[], y: NoInfer<T>) {
    return x.concat(y);
}

const r2 = fn2(['a', 'b'], 'c'); // Lỗi: Đối số kiểu '"c"'
không thể gán cho tham số có kiểu '"a" | "b"'.
```

Khác

Lỗi và xử lý ngoại lệ

TypeScript cho phép bạn bắt và xử lý lỗi bằng các cơ chế xử lý lỗi JavaScript tiêu chuẩn:

Khối Try-Catch-Finally:

```
try {
    // Mã có thể ném ra lỗi
} catch (error) {
    // Xử lý lỗi
} finally {
    // Mã luôn được thực thi, finally là tùy chọn
}
```

Bạn cũng có thể xử lý các loại lỗi khác nhau:

```
try {
    // Mã có thể ném ra các loại lỗi khác nhau
} catch (error) {
    if (error instanceof TypeError) {
        // Xử lý TypeError
    } else if (error instanceof RangeError) {
        // Xử lý RangeError
    } else {
        // Xử lý các lỗi khác
    }
}
```

Kiểu lỗi tùy chỉnh:

Có thể chỉ định các lỗi cụ thể hơn bằng cách mở rộng `class` Error:

```
class CustomError extends Error {
    constructor(message: string) {
        super(message);
        this.name = 'CustomError';
    }
}

throw new CustomError('This is a custom error.');
```

Mixin class

Mixin class cho phép bạn kết hợp và tổng hợp hành vi từ nhiều class vào một class duy nhất. Chúng cung cấp cách tái sử dụng và mở rộng chức năng mà không cần các chuỗi kế thừa sâu.

```
abstract class Identifiable {
    name: string = '';
    logId() {
        console.log('id:', this.name);
    }
}
abstract class Selectable {
    selected: boolean = false;
    select() {
        this.selected = true;
        console.log('Select');
    }
    deselect() {
        this.selected = false;
        console.log('Deselect');
    }
}
class MyClass {
    constructor() {}
}

// Mở rộng MyClass để bao gồm hành vi của Identifiable và
Selectable
interface MyClass extends Identifiable, Selectable {}

// Hàm để áp dụng mixin cho một lớp
function applyMixins(source: any, baseCtors: any[]) {
    baseCtors.forEach(baseCtor => {

Object.getOwnPropertyNames(baseCtor.prototype).forEach(name   =>
{
            let descriptor = Object.getOwnPropertyDescriptor(
```

```
                baseCtor.prototype,
                name
            );
            if (descriptor) {
                Object.defineProperty(source.prototype, name,
descriptor);
            }
        });
    });
}

// Áp dụng mixin cho MyClass
applyMixins(MyClass, [Identifiable, Selectable]);
let o = new MyClass();
o.name = 'abc';
o.logId();
o.select();
```

Các tính năng ngôn ngữ bất đồng bộ

Vì TypeScript là tập cha của JavaScript, nó có các tính năng ngôn ngữ bất đồng bộ tích hợp của JavaScript như:

Promise:

Promise là cách xử lý các thao tác bất đồng bộ và kết quả của chúng bằng các phương thức như `.then()` và `.catch()` để xử lý điều kiện thành công và lỗi.

Tìm hiểu thêm: https://developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Global_Objects/Promise

Async/await:

Các từ khóa Async/await cung cấp cú pháp trông đồng bộ hơn khi làm việc với Promise. Từ khóa `async` được dùng để định

nghĩa một hàm bất đồng bộ và từ khóa `await` được dùng bên trong hàm async để tạm dừng thực thi cho đến khi Promise được resolve hoặc reject.

Tìm hiểu thêm: https://developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Statements/async_function https://developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Operators/await

Các API sau được TypeScript hỗ trợ tốt:

Fetch API: https://developer.mozilla.org/en-US/docs/Web/API/Fetch_API

Web Workers: https://developer.mozilla.org/en-US/docs/Web/API/Web_Workers_API

Shared Workers: https://developer.mozilla.org/en-US/docs/Web/API/SharedWorker

WebSocket: https://developer.mozilla.org/en-US/docs/Web/API/WebSockets_API

Iterator và Generator

Cả Iterator và Generator đều được TypeScript hỗ trợ tốt.

Iterator là các đối tượng triển khai iterator protocol, cung cấp cách truy cập từng phần tử của một collection hoặc sequence. Nó là một cấu trúc chứa con trỏ đến phần tử tiếp theo trong quá trình lặp. Chúng có phương thức `next()` trả về giá trị tiếp theo trong sequence cùng một giá trị boolean cho biết sequence đã `done` hay chưa.

```
class NumberIterator implements Iterable<number> {
    private current: number;

    constructor(
        private start: number,
        private end: number
    ) {
        this.current = start;
    }

    public next(): IteratorResult<number> {
        if (this.current <= this.end) {
            const value = this.current;
            this.current++;
            return { value, done: false };
        } else {
            return { value: undefined, done: true };
        }
    }

    [Symbol.iterator](): Iterator<number> {
        return this;
    }
}

const iterator = new NumberIterator(1, 3);

for (const num of iterator) {
    console.log(num);
}
```

Generator là các hàm đặc biệt được định nghĩa bằng cú pháp `function*` giúp đơn giản hóa việc tạo iterator. Chúng sử dụng từ khóa `yield` để định nghĩa chuỗi giá trị và tự động tạm dừng rồi tiếp tục thực thi khi giá trị được yêu cầu.

Generator giúp tạo iterator dễ dàng hơn và đặc biệt hữu ích khi làm việc với sequence lớn hoặc vô hạn.

Ví dụ:

```
function* numberGenerator(start: number, end: number): Generator<number> {
    for (let i = start; i <= end; i++) {
        yield i;
    }
}

const generator = numberGenerator(1, 5);

for (const num of generator) {
    console.log(num);
}
```

TypeScript cũng hỗ trợ async iterator và async Generator.

Tìm hiểu thêm:

https://developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Global_Objects/Generator

https://developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Global_Objects/Iterator

Tham khảo TsDocs JSDoc

Khi làm việc với codebase JavaScript, có thể giúp TypeScript suy luận đúng kiểu bằng cách sử dụng comment JSDoc với annotation bổ sung để cung cấp thông tin kiểu.

Ví dụ:

```
/**
 * Tính lũy thừa của một số cho trước
 * @constructor
 * @param {number} base - Giá trị cơ số của biểu thức
 * @param {number} exponent - Giá trị số mũ của biểu thức
 */
function power(base: number, exponent: number) {
    return Math.pow(base, exponent);
}
power(10, 2); // function power(base: number, exponent: number): number
```

Tài liệu đầy đủ có tại liên kết này: https://www.typescriptlang.org/docs/handbook/jsdoc-supported-types.html

Từ phiên bản 3.7, có thể tạo định nghĩa kiểu .d.ts từ cú pháp JavaScript JSDoc. Bạn có thể tìm thêm thông tin tại đây: https://www.typescriptlang.org/docs/handbook/declaration-files/dts-from-js.html

@types

Các package thuộc tổ chức @types sử dụng quy ước đặt tên package đặc biệt để cung cấp định nghĩa kiểu cho các thư viện hoặc module JavaScript hiện có. Ví dụ, sử dụng:

```
npm install --save-dev @types/lodash
```

sẽ cài đặt các định nghĩa kiểu của `lodash` vào dự án hiện tại của bạn.

Để đóng góp cho các định nghĩa kiểu của một package `@types`, hãy gửi pull request tới https://github.com/DefinitelyTyped/DefinitelyTyped.

JSX

JSX (JavaScript XML) là một phần mở rộng cú pháp của ngôn ngữ JavaScript cho phép bạn viết mã giống HTML trong các tệp JavaScript hoặc TypeScript. Nó thường được dùng trong React để định nghĩa cấu trúc HTML.

TypeScript mở rộng khả năng của JSX bằng cách cung cấp kiểm tra kiểu và phân tích tĩnh.

Để sử dụng JSX, bạn cần đặt tùy chọn trình biên dịch `jsx` trong tệp `tsconfig.json`. Hai tùy chọn cấu hình phổ biến:

- "preserve": emit tệp .jsx với JSX không thay đổi. Tùy chọn này yêu cầu TypeScript giữ nguyên cú pháp JSX và không biến đổi nó trong quá trình biên dịch. Bạn có thể dùng tùy chọn này nếu có công cụ riêng như Babel xử lý việc biến đổi.
- "react": bật phép biến đổi JSX tích hợp của TypeScript. React.createElement sẽ được sử dụng.

Tất cả tùy chọn có tại đây: https://www.typescriptlang.org/tsconfig#jsx

ES6 Module

TypeScript hỗ trợ ES6 (ECMAScript 2015) và nhiều phiên bản tiếp theo. Điều này có nghĩa bạn có thể sử dụng cú pháp ES6 như arrow function, template literal, class, module, destructuring và nhiều tính năng khác.

Để bật các tính năng ES6 trong dự án, bạn có thể chỉ định thuộc tính `target` trong tsconfig.json.

Ví dụ cấu hình:

```
{
  "compilerOptions": {
    "target": "es6",
    "module": "es6",
    "moduleResolution": "node",
    "sourceMap": true,
    "outDir": "dist"
  },
  "include": ["src"]
}
```

Toán tử lũy thừa ES7

Toán tử lũy thừa (`**`) tính giá trị bằng cách nâng toán hạng thứ nhất lên lũy thừa của toán hạng thứ hai. Nó hoạt động tương tự `Math.pow()` nhưng có thêm khả năng chấp nhận BigInt làm toán hạng. TypeScript hỗ trợ đầy đủ toán tử này bằng cách đặt `target` trong tệp tsconfig.json thành `es2016` hoặc cao hơn.

```
console.log(2 ** (2 ** 2)); // 16
```

Câu lệnh for-await-of

Đây là một tính năng JavaScript được TypeScript hỗ trợ đầy đủ, cho phép bạn lặp qua các đối tượng iterable bất đồng bộ với phiên bản target `es2018`.

```
async function* asyncNumbers(): AsyncIterableIterator<number> {
    yield Promise.resolve(1);
    yield Promise.resolve(2);
    yield Promise.resolve(3);
}

(async () => {
```

```
    for await (const num of asyncNumbers()) {
        console.log(num);
    }
})();
```

Meta-property new target

Bạn có thể dùng meta-property `new.target` trong TypeScript, cho phép xác định một hàm hoặc constructor có được gọi bằng toán tử new hay không. Nó cho phép phát hiện liệu một đối tượng có được tạo ra do lời gọi constructor hay không.

```
class Parent {
    constructor() {
          console.log(new.target); // In ra hàm khởi tạo được
dùng để tạo thể hiện
    }
}

class Child extends Parent {
    constructor() {
        super();
    }
}

const parentX = new Parent(); // [Function: Parent]
const child = new Child(); // [Function: Child]
```

Biểu thức Dynamic Import

Có thể tải module có điều kiện hoặc lazy-load chúng theo nhu cầu bằng đề xuất ECMAScript cho dynamic import, được TypeScript hỗ trợ.

Cú pháp cho biểu thức dynamic import trong TypeScript như sau:

```
async function renderWidget() {
    const container = document.getElementById('widget');
    if (container !== null) {
        const widget = await import('./widget'); // Nhập động
        widget.render(container);
    }
}

renderWidget();
```

“tsc –watch”

Lệnh này khởi động trình biên dịch TypeScript với tham số `--watch`, có khả năng tự động biên dịch lại các tệp TypeScript mỗi khi chúng được sửa đổi.

```
tsc --watch
```

Bắt đầu từ TypeScript phiên bản 4.9, việc theo dõi tệp chủ yếu dựa vào sự kiện hệ thống tệp và tự động chuyển sang polling nếu không thể thiết lập watcher dựa trên sự kiện.

Toán tử Non-null Assertion

Toán tử non-null assertion (postfix !), còn gọi là definite assignment assertion, là một tính năng TypeScript cho phép bạn assertion rằng một biến hoặc thuộc tính không phải null hoặc undefined, ngay cả khi phân tích kiểu tĩnh của TypeScript cho rằng nó có thể như vậy. Với tính năng này, có thể loại bỏ mọi kiểm tra tường minh.

```
type Person = {
    name: string;
};

const printName = (person?: Person) => {
```

```
    console.log(`Name is ${person!.name}`);
};
```

Khai báo có giá trị mặc định

Defaulted declaration được dùng khi một biến hoặc tham số được gán giá trị mặc định. Điều này có nghĩa nếu không cung cấp giá trị cho biến hoặc tham số đó, giá trị mặc định sẽ được dùng thay thế.

```
function greet(name: string = 'Anonymous'): void {
    console.log(`Hello, ${name}!`);
}
greet(); // Hello, Anonymous!
greet('John'); // Hello, John!
```

Optional Chaining

Toán tử optional chaining `?.` hoạt động giống toán tử dấu chấm thông thường (`.`) để truy cập thuộc tính hoặc phương thức. Tuy nhiên, nó xử lý null hoặc undefined một cách an toàn bằng cách kết thúc biểu thức và trả về `undefined` thay vì throw lỗi.

```
type Person = {
    name: string;
    age?: number;
    address?: {
        street?: string;
        city?: string;
    };
};

const person: Person = {
    name: 'John',
};
```

```
console.log(person.address?.city); // undefined
```

Toán tử Nullish Coalescing

Toán tử nullish coalescing `??` trả về giá trị bên phải nếu giá trị bên trái là `null` hoặc `undefined`; nếu không, nó trả về giá trị bên trái.

```
const foo = null ?? 'foo';
console.log(foo); // foo

const baz = 1 ?? 'baz';
const baz2 = 0 ?? 'baz';
console.log(baz); // 1
console.log(baz2); // 0
```

Template Literal Type

Template Literal Type cho phép bạn thao tác giá trị chuỗi ở cấp độ kiểu và tạo các string type mới dựa trên các kiểu hiện có. Chúng hữu ích để tạo các kiểu biểu đạt và chính xác hơn từ các thao tác dựa trên chuỗi.

```
type Department = 'engineering' | 'hr';
type Language = 'english' | 'spanish';
type Id = `${Department}-${Language}-id`; // "engineering-english-id" | "engineering-spanish-id" | "hr-english-id" | "hr-spanish-id"
```

Function overloading

Function overloading cho phép bạn định nghĩa nhiều chữ ký hàm cho cùng một tên hàm, mỗi chữ ký có kiểu tham số và kiểu trả về khác nhau. Khi bạn gọi một hàm được overload,

TypeScript sử dụng các đối số được cung cấp để xác định chữ ký hàm đúng:

```
function makeGreeting(name: string): string;
function makeGreeting(names: string[]): string[];

function makeGreeting(person: unknown): unknown {
    if (typeof person === 'string') {
        return `Hi ${person}!`;
    } else if (Array.isArray(person)) {
        return person.map(name => `Hi, ${name}!`);
    }
    throw new Error('Unable to greet');
}

makeGreeting('Simon');
makeGreeting(['Simone', 'John']);
```

Kiểu đệ quy

Recursive Type là một kiểu có thể tham chiếu đến chính nó. Điều này hữu ích để định nghĩa các cấu trúc dữ liệu có cấu trúc phân cấp hoặc đệ quy (có khả năng lồng vô hạn), chẳng hạn linked list, tree và graph.

```
type ListNode<T> = {
    data: T;
    next: ListNode<T> | undefined;
};
```

Recursive Conditional Type

Có thể định nghĩa các quan hệ kiểu phức tạp bằng logic và đệ quy trong TypeScript. Hãy phân tích theo cách đơn giản:

Conditional Type cho phép bạn định nghĩa kiểu dựa trên điều kiện boolean:

```
type CheckNumber<T> = T extends number ? 'Number' : 'Not a number';
type A = CheckNumber<123>; // 'Number'
type B = CheckNumber<'abc'>; // 'Not a number'
```

Đệ quy nghĩa là một định nghĩa kiểu tham chiếu đến chính nó trong định nghĩa của mình:

```
type Json = string | number | boolean | null | Json[] | { [key: string]: Json };

const data: Json = {
    prop1: true,
    prop2: 'prop2',
    prop3: {
        prop4: [],
    },
};
```

Recursive Conditional Type kết hợp cả logic điều kiện và đệ quy. Điều này có nghĩa một định nghĩa kiểu có thể phụ thuộc vào chính nó thông qua logic điều kiện, tạo ra các quan hệ kiểu phức tạp và linh hoạt.

```
type Flatten<T> = T extends Array<infer U> ? Flatten<U> : T;

type NestedArray = [1, [2, [3, 4], 5], 6];
type FlattenedArray = Flatten<NestedArray>; // 2 | 3 | 4 | 5 | 1 | 6
```

Hỗ trợ ECMAScript Module trong Node

Node.js bổ sung hỗ trợ ECMAScript Module bắt đầu từ phiên bản 15.3.0 và TypeScript đã hỗ trợ ECMAScript Module cho

Node.js kể từ phiên bản 4.7. Có thể bật hỗ trợ này bằng cách sử dụng thuộc tính `module` với giá trị `nodenext` trong tệp tsconfig.json. Dưới đây là một ví dụ:

```
{
  "compilerOptions": {
    "module": "nodenext",
    "outDir": "./lib",
    "declaration": true
  }
}
```

Node.js hỗ trợ hai phần mở rộng tệp cho module: `.mjs` cho ES module và `.cjs` cho CommonJS module. Các phần mở rộng tệp tương ứng trong TypeScript là `.mts` cho ES module và `.cts` cho CommonJS module. Khi trình biên dịch TypeScript chuyển dịch các tệp này sang JavaScript, nó sẽ tạo các tệp `.mjs` và `.cjs`.

Nếu muốn sử dụng ES module trong dự án, bạn có thể đặt thuộc tính `type` thành “module” trong tệp package.json. Điều này yêu cầu Node.js xử lý dự án như một dự án ES module.

Ngoài ra, TypeScript cũng hỗ trợ khai báo kiểu trong các tệp .d.ts. Các tệp khai báo này cung cấp thông tin kiểu cho thư viện hoặc module viết bằng TypeScript, cho phép các lập trình viên khác sử dụng chúng với tính năng kiểm tra kiểu và tự động hoàn thành của TypeScript.

Assertion Function

Trong TypeScript, assertion function là các hàm biểu thị việc xác minh một điều kiện cụ thể dựa trên giá trị trả về của chúng. Ở dạng đơn giản nhất, một hàm assert kiểm tra predicate được cung cấp và phát sinh lỗi khi predicate được đánh giá là false.

```
function isNumber(value: unknown): asserts value is number {
    if (typeof value !== 'number') {
        throw new Error('Not a number');
    }
}
```

Hoặc có thể được khai báo như một function expression:

```
type AssertIsNumber = (value: unknown) => asserts value is
number;
const isNumber: AssertIsNumber = value => {
    if (typeof value !== 'number') {
        throw new Error('Not a number');
    }
};
```

Assertion function có điểm tương đồng với type guard. Type guard ban đầu được giới thiệu để thực hiện kiểm tra thời gian chạy và bảo đảm kiểu của một giá trị trong một phạm vi cụ thể. Cụ thể, type guard là một hàm đánh giá type predicate và trả về giá trị boolean cho biết predicate là true hay false. Điều này hơi khác assertion function, trong đó mục đích là throw lỗi thay vì trả về false khi predicate không được thỏa mãn.

Ví dụ về type guard:

```
const isNumber = (value: unknown): value is number => typeof
value === 'number';
```

Variadic Tuple Type

Variadic Tuple Type là một tính năng được giới thiệu trong TypeScript phiên bản 4.0, vì vậy trước tiên hãy nhắc lại tuple là gì:

Tuple type là một mảng có độ dài được định nghĩa và kiểu của từng phần tử đã biết:

```
type Student = [string, number];
const [name, age]: Student = ['Simone', 20];
```

Thuật ngữ "variadic" có nghĩa là arity không xác định (chấp nhận số lượng đối số thay đổi).

Variadic tuple là một tuple type có tất cả thuộc tính như trước nhưng hình dạng chính xác chưa được định nghĩa:

```
type Bar<T extends unknown[]> = [boolean, ...T, number];

type A = Bar<[boolean]>; // [boolean, boolean, number]
type B = Bar<['a', 'b']>; // [boolean, 'a', 'b', number]
type C = Bar<[]>; // [boolean, number]
```

Trong đoạn mã trước, chúng ta có thể thấy hình dạng tuple được định nghĩa bởi generic `T` được truyền vào.

Variadic tuple có thể chấp nhận nhiều generic, giúp chúng rất linh hoạt:

```
type Bar<T extends unknown[], G extends unknown[]> = [...T,
boolean, ...G];

type A = Bar<[number], [string]>; // [number, boolean, string]
type B = Bar<['a', 'b'], [boolean]>; // ["a", "b", boolean,
boolean]
```

Với variadic tuple mới, chúng ta có thể sử dụng:

- Spread trong cú pháp tuple type giờ có thể là generic, nhờ đó chúng ta có thể biểu diễn các thao tác higher-order trên

tuple và array ngay cả khi chưa biết các kiểu thực tế đang thao tác.
- Rest element có thể xuất hiện ở bất kỳ đâu trong tuple.

Ví dụ:

```
type Items = readonly unknown[];

function concat<T extends Items, U extends Items>(
    arr1: T,
    arr2: U
): [...T, ...U] {
    return [...arr1, ...arr2];
}

concat([1, 2, 3], ['4', '5', '6']); // [1, 2, 3, "4", "5", "6"]
```

Boxed type

Boxed type nói đến các wrapper object được dùng để biểu diễn primitive type dưới dạng object. Các wrapper object này cung cấp chức năng và phương thức bổ sung không có trực tiếp trên các giá trị primitive.

Khi bạn truy cập một phương thức như `charAt` hoặc `normalize` trên primitive `string`, JavaScript bọc nó trong một đối tượng `String`, gọi phương thức rồi loại bỏ đối tượng.

Minh họa:

```
const originalNormalize = String.prototype.normalize;
String.prototype.normalize = function () {
    console.log(this, typeof this);
    return originalNormalize.call(this);
};
console.log('\u0041'.normalize());
```

TypeScript biểu diễn sự khác biệt này bằng cách cung cấp các kiểu riêng cho primitive và object wrapper tương ứng:

- string => String
- number => Number
- boolean => Boolean
- symbol => Symbol
- bigint => BigInt

Boxed type thường không cần thiết. Tránh sử dụng boxed type và thay vào đó sử dụng primitive type, chẳng hạn `string` thay vì `String`.

Covariance và Contravariance trong TypeScript

Covariance và contravariance mô tả quan hệ giữa các kiểu bên trong kiểu generic. `Dog` là một `Animal`, nhưng không phải mọi `Animal` đều là `Dog`.

Covariance (tạo giá trị): hàm trả về `Dog` có thể thay thế hàm trả về `Animal`. Mọi con chó đều là động vật, nhưng hàm trả về động vật bất kỳ có thể trả về mèo.

```
class Animal {
    name = '';
}

class Dog extends Animal {
    breed = '';
}

type Producer<T> = () => T;

declare let produceAnimal: Producer<Animal>;
declare let produceDog: Producer<Dog>;
```

```
produceAnimal = produceDog; // Hợp lệ
produceDog = produceAnimal; // Lỗi
```

Contravariance (nhận giá trị): hàm nhận mọi `Animal` có thể thay thế hàm chỉ nhận `Dog`. Hàm đó xử lý được cả chó; hàm chỉ nhận chó không thể xử lý an toàn mọi động vật.

```
type Consumer<T> = (value: T) => void;

declare let consumeAnimal: Consumer<Animal>;
declare let consumeDog: Consumer<Dog>;

consumeDog = consumeAnimal; // Hợp lệ
consumeAnimal = consumeDog; // Lỗi khi dùng strictFunctionTypes
```

Mũi tên cho biết hướng gán an toàn:

- Dog -> Animal
- Producer<Dog> -> Producer<Animal> (covariance)
- Consumer<Animal> -> Consumer<Dog> (contravariance)

Mảng có thể thay đổi: TypeScript cũng cho phép gán `Dog[]` cho `Animal[]`. Hai biến cùng tham chiếu một mảng; thêm `Animal` qua kiểu rộng hơn có thể đưa một phần tử không phải chó vào mảng chó.

```
const dogs: Dog[] = [new Dog()];
const animals: Animal[] = dogs;

animals.push(new Animal()); // Được phép, nhưng không an toàn
// dogs hiện chứa một Animal không phải là Dog.
```

Tùy chọn trình biên dịch: khi bật `strictFunctionTypes`, tham số của hàm độc lập được kiểm tra theo contravariance. Nếu tắt, TypeScript có thể cho phép cả hai hướng (bivariance) dù không

an toàn. Tham số trong khai báo phương thức và hàm tạo là ngoại lệ.

Variance Annotation tùy chọn cho tham số kiểu

Từ TypeScript 4.7, `out` biểu thị covariance, `in` biểu thị contravariance và `in out` biểu thị invariance. TypeScript thường tự suy luận variance. Chú thích phải phù hợp với cách dùng tham số generic, không tùy ý thay đổi hành vi của nó.

```
type Producer<out T> = () => T;
type Consumer<in T> = (value: T) => void;
type Transformer<in out T> = (value: T) => T;
```

Template String Pattern Index Signature

Template string pattern index signature cho phép chúng ta định nghĩa index signature linh hoạt bằng template string pattern. Tính năng này cho phép tạo đối tượng có thể được index bằng các pattern cụ thể của string key, mang lại nhiều kiểm soát và tính cụ thể hơn khi truy cập và thao tác thuộc tính.

Từ phiên bản 4.4, TypeScript cho phép index signature cho symbol và template string pattern.

```
const uniqueSymbol = Symbol('description');

type MyKeys = `key-${string}`;

type MyObject = {
    [uniqueSymbol]: string;
    [key: MyKeys]: number;
};

const obj: MyObject = {
```

```
    [uniqueSymbol]: 'Unique symbol key',
    'key-a': 123,
    'key-b': 456,
};

console.log(obj[uniqueSymbol]); // Unique symbol key
console.log(obj['key-a']); // 123
console.log(obj['key-b']); // 456
```

Toán tử satisfies

Toán tử `satisfies` cho phép bạn kiểm tra một kiểu đã cho có thỏa mãn một interface hoặc điều kiện cụ thể hay không. Nói cách khác, nó bảo đảm một kiểu có tất cả thuộc tính và phương thức bắt buộc của một interface cụ thể. Đây là cách để bảo đảm một biến phù hợp với định nghĩa của một kiểu. Dưới đây là một ví dụ:

```
type Columns = 'name' | 'nickName' | 'attributes';

type User = Record<Columns, string | string[] | undefined>;

// Chú thích kiểu bằng `User`
const user: User = {
    name: 'Simone',
    nickName: undefined,
    attributes: ['dev', 'admin'],
};

// Trong các dòng sau, TypeScript không thể suy luận chính xác
user.attributes?.map(console.log); // Thuộc tính 'map' không
tồn tại trên kiểu 'string | string[]'. Thuộc tính 'map' không
tồn tại trên kiểu 'string'.
user.nickName; // string | string[] | undefined

// Khẳng định kiểu bằng `as`
const user2 = {
```

```
    name: 'Simon',
    nickName: undefined,
    attributes: ['dev', 'admin'],
} as User;

// Ở đây cũng vậy, TypeScript không thể suy luận chính xác
user2.attributes?.map(console.log); // Thuộc tính 'map' không tồn tại trên kiểu 'string | string[]'. Thuộc tính 'map' không tồn tại trên kiểu 'string'.
user2.nickName; // string | string[] | undefined

// Với toán tử `satisfies`, giờ ta có thể suy luận kiểu chính xác
const user3 = {
    name: 'Simon',
    nickName: undefined,
    attributes: ['dev', 'admin'],
} satisfies User;

user3.attributes?.map(console.log); // TypeScript suy luận chính xác: string[]
user3.nickName; // TypeScript suy luận chính xác: undefined
```

Import và Export chỉ dành cho Type

Type-Only Import và Export cho phép bạn import hoặc export type mà không import hoặc export các giá trị hay hàm gắn với các type đó. Điều này có thể hữu ích để giảm kích thước bundle.

Để sử dụng type-only import, bạn có thể dùng từ khóa `import type`.

TypeScript cho phép sử dụng cả phần mở rộng tệp khai báo và triển khai (.ts, .mts, .cts và .tsx) trong type-only import, bất kể thiết lập `allowImportingTsExtensions`.

Ví dụ:

```ts
import type { House } from './house.ts';
```

Các dạng sau được hỗ trợ:

```ts
import type T from './mod';
import type { A, B } from './mod';
import type * as Types from './mod';
export type { T };
export type { T } from './mod';
```

Khai báo using và Explicit Resource Management

Khai báo `using` là một binding bất biến có phạm vi block, tương tự `const`, được dùng để quản lý các resource có thể dispose. Khi được khởi tạo bằng một giá trị, phương thức `Symbol.dispose` của giá trị đó được ghi lại và sau đó được thực thi khi thoát khỏi block scope bao quanh.

Điều này dựa trên tính năng Resource Management của ECMAScript, hữu ích để thực hiện các tác vụ cleanup thiết yếu sau khi tạo đối tượng, chẳng hạn đóng connection, xóa tệp và giải phóng bộ nhớ.

Ghi chú:

- Do mới được giới thiệu trong TypeScript phiên bản 5.2, hầu hết runtime chưa hỗ trợ native. Bạn sẽ cần polyfill cho: `Symbol.dispose`, `Symbol.asyncDispose`, `DisposableStack`, `AsyncDisposableStack`, `SuppressedError`.
- Ngoài ra, bạn cần cấu hình tsconfig.json như sau:

```ts
{
    "compilerOptions": {
        "target": "es2022",
        "lib": ["es2022", "esnext.disposable", "dom"]
```

```
    }
}
```

Ví dụ:

```
//@ts-ignore
Symbol.dispose ??= Symbol('Symbol.dispose'); // Polyfill đơn giản

const doWork = (): Disposable => {
    return {
        [Symbol.dispose]: () => {
            console.log('disposed');
        },
    };
};

console.log(1);

{
    using work = doWork(); // Tài nguyên được khai báo
    console.log(2);
} // Tài nguyên được giải phóng (ví dụ: `work[Symbol.dispose]()` được thực thi)

console.log(3);
```

Mã sẽ ghi ra:

```
1
2
disposed
3
```

Một resource đủ điều kiện để dispose phải tuân theo interface `Disposable`:

```
// lib.esnext.disposable.d.ts
interface Disposable {
    [Symbol.dispose](): void;
}
```

Khai báo `using` ghi các thao tác dispose resource vào một stack, bảo đảm chúng được dispose theo thứ tự ngược với thứ tự khai báo:

```
{
    using j = getA(),
        y = getB();
    using k = getC();
} // giải phóng `C`, rồi `B`, rồi `A`.
```

Resource được bảo đảm sẽ được dispose ngay cả khi mã tiếp theo hoặc exception xảy ra. Điều này có thể dẫn đến việc dispose phát sinh exception và có khả năng suppress một exception khác. Để giữ lại thông tin về các lỗi bị suppress, một native exception mới, `SuppressedError`, được giới thiệu.

Khai báo await using

Khai báo `await using` xử lý một resource được dispose bất đồng bộ. Giá trị phải có phương thức `Symbol.asyncDispose`, phương thức này sẽ được await ở cuối block.

```
async function doWorkAsync() {
     await using work = doWorkAsync(); // Tài nguyên được khai
báo
}   // Tài nguyên được giải phóng (ví dụ: `await
work[Symbol.asyncDispose]()` được thực thi)
```

Với resource được dispose bất đồng bộ, nó phải tuân theo interface `Disposable` hoặc `AsyncDisposable`:

```
// lib.esnext.disposable.d.ts
interface AsyncDisposable {
    [Symbol.asyncDispose](): Promise<void>;
}

//@ts-ignore
Symbol.asyncDispose ??= Symbol('Symbol.asyncDispose'); // Polyfill đơn giản

class DatabaseConnection implements AsyncDisposable {
    // Phương thức được gọi khi đối tượng được giải phóng bất đồng bộ
    [Symbol.asyncDispose]() {
        // Đóng kết nối và trả về một promise
        return this.close();
    }

    async close() {
        console.log('Closing the connection...');
            await new Promise(resolve => setTimeout(resolve, 1000));
        console.log('Connection closed.');
    }
}

async function doWork() {
    // Tạo kết nối mới và giải phóng bất đồng bộ khi ra khỏi phạm vi
    await using connection = new DatabaseConnection(); // Tài nguyên được khai báo
    console.log('Doing some work...');
} // Tài nguyên được giải phóng (ví dụ: `await connection[Symbol.asyncDispose]()` được thực thi)

doWork();
```

Mã ghi ra:

```
Doing some work...
Closing the connection...
Connection closed.
```

Các khai báo `using` và `await using` được phép trong các câu lệnh: `for`, `for-in`, `for-of`, `for-await-of`, `switch`.

Import Attribute

Import Attribute của TypeScript 5.3 (nhãn cho import) cho runtime biết cách xử lý module (JSON, v.v.). Điều này cải thiện bảo mật bằng cách bảo đảm import rõ ràng và phù hợp với Content Security Policy (CSP) để tải resource an toàn hơn. TypeScript bảo đảm chúng hợp lệ nhưng để runtime xử lý cách diễn giải cho việc xử lý module cụ thể.

Ví dụ:

```
import config from './config.json' with { type: 'json' };
```

với dynamic import:

```
const config = import('./config.json', { with: { type: 'json' }
});
```

Kiểm tra cú pháp Regular Expression

Kể từ TypeScript 5.5.4, TypeScript kiểm tra regex literal để phát hiện các lỗi phổ biến tại thời điểm biên dịch (ví dụ cú pháp không hợp lệ, backreference sai, tính năng không được hỗ trợ cho phiên bản JS target). Điều này giúp phát hiện bug sớm hơn nhưng không kiểm tra chuỗi new RegExp(“...”).

```
let r = /(a)\2/; // Lỗi: Tham chiếu ngược này đề cập đến một
nhóm không tồn tại.
```

import defer

`import defer` cho phép bạn tải một module nhưng trì hoãn việc thực thi cho đến khi thực sự sử dụng thứ gì đó từ module. Điều này giúp tránh công việc và side effect không cần thiết.

- Chỉ hoạt động với: `import defer * as name from "module"`
- Mã chỉ chạy khi bạn truy cập một export